

C# Komplett

Ali Ilci

Einleitung

- Kursablauf & Organisatorisches
- Räumlichkeiten
- Vorstellungsrunde
  - Hintergrund, Motivation und Vorkenntnisse
- Bei Unklarheiten: Möglichst sofort fragen!

Über mich – Ali Ilci

- Freiberuflicher IT - Trainer (MCT)
- Fachinformatiker
- Erziehungswissenschaftler
- Themenschwerpunkte:
  - ASP.NET Core / MVC / WebApi / Blazor
  - C#
  - Html5 & Css3
  - JavaScript

- www.ilci.de



Themenwolke

LinQ

Werte- /Referenztypen

Interfaces

.NET Core Framework

Delegates

OOP

Polymorphismus

Entity Framework

C# Grundlagen

CLR / BCL

Generics

Typen / Kontrollstrukturen

Klassen

Vererbung

Visual Studio

Neu in C#

Statisch

Konstruktoren

Überladung

Exception

Erweiterungsmethoden

Events

Collections

Kapselung

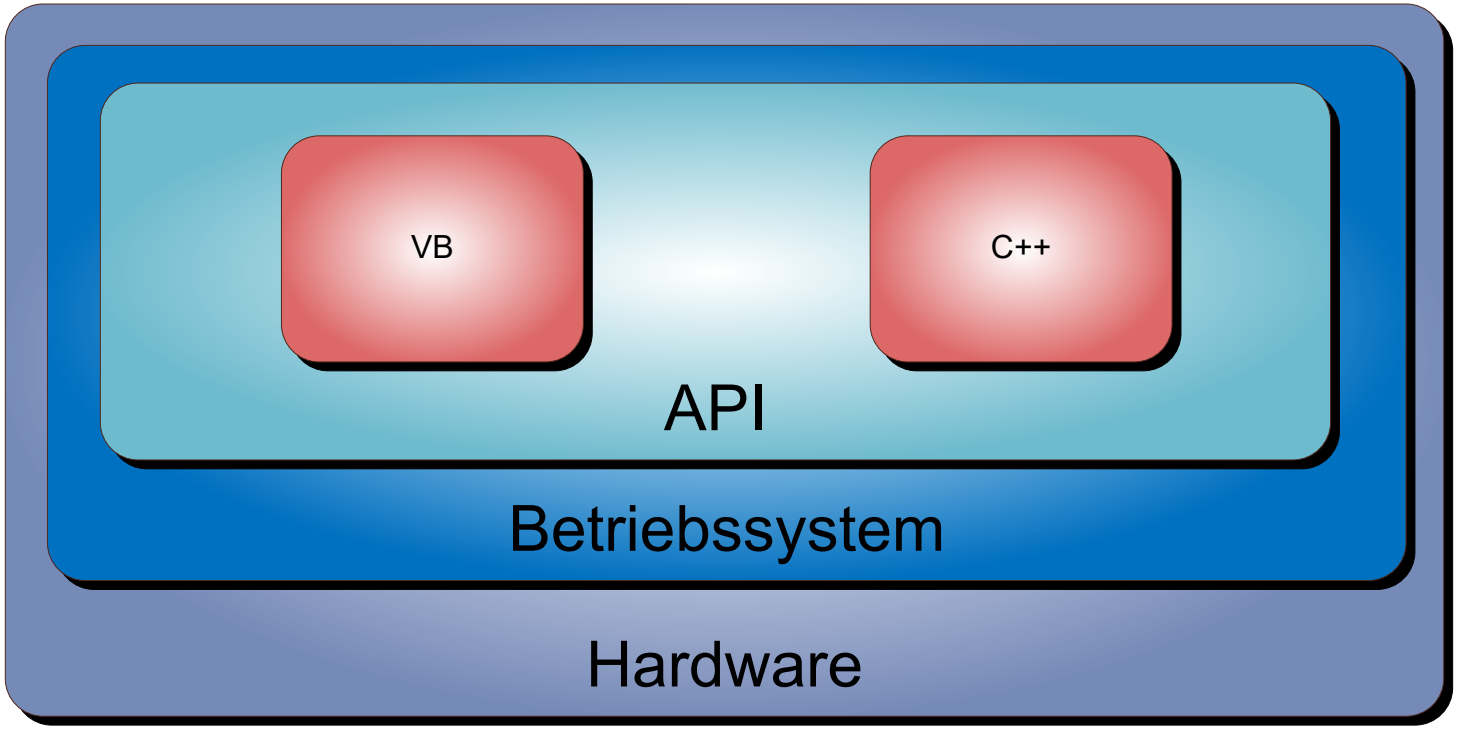
Überblick .NET

.NET Core

Themen

- .NET Core Framework
- Entwicklungsumgebung Visual Studio

Vor .NET



Probleme vor .NET

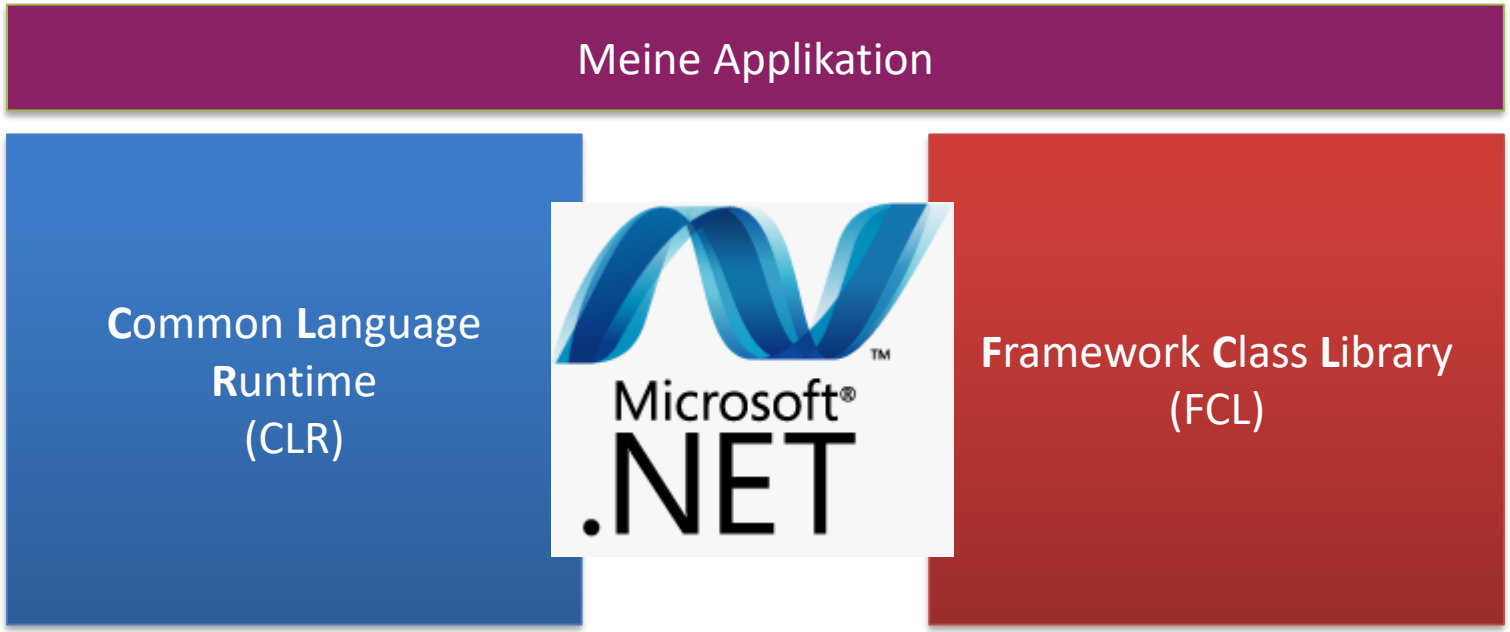
- **Umfang**
  - Erweiterung Betriebssystem -> API -> Sprache
  - Wachstum der jeweiligen (!) Sprache
- **Inkompatibilität**
  - Datentypen nicht austauschbar (in Sprachen eingebettet)
  - Sprachen (funktional) inkompatibel
- **Plattformabhängigkeit**
  - Programme an Betriebssystem gebunden
  - Kein unmittelbarer Austausch von Daten

.NET Framework

*„**.NET** [ˈdɔtnɛt] bezeichnet eine von Microsoft entwickelte **Software-Plattform** zur Entwicklung und Ausführung von **Anwendungsprogrammen**. .NET besteht aus einer **Laufzeitumgebung** (Common Language Runtime), in der die Programme ausgeführt werden, sowie einer Sammlung von **Klassenbibliotheken**, **Programmierschnittstellen** und **Dienstprogrammen** (Services).“*

Quelle: Wikipedia

.NET Duo


Meine Applikation
Common Language Runtime (CLR)
Microsoft® .NET
Framework Class Library (FCL)

Common Language Runtime

- Laufzeitumgebung des .NET Frameworks
- Verwaltet die Ausführung von .NET Programmen
  - Speicher, Thread- und Codeausführung
- Bereitstellung von Basisdiensten
  - Compilierung, Code Access Security, Garbage Collector
- Die CLR ist eine einheitliche Laufzeitumgebung unabhängig von der genutzten Programmiersprache, dem Betriebssystem und Hardware

Common Type System

- Problem:
  - Datentypen sind in verschiedenen Sprachen nicht identisch
    - Bsp: Integer -> 16 oder 32Bit?
- CTS garantiert Datentypsicherheit innerhalb von .NET Sprachen (gemeinsame Schnittmenge)
- Sprachen sind interoperabel
  - Bsp: Klasse in C# kann von einer VB.NET-Klasse erben

Managed vs. Unmanaged Code

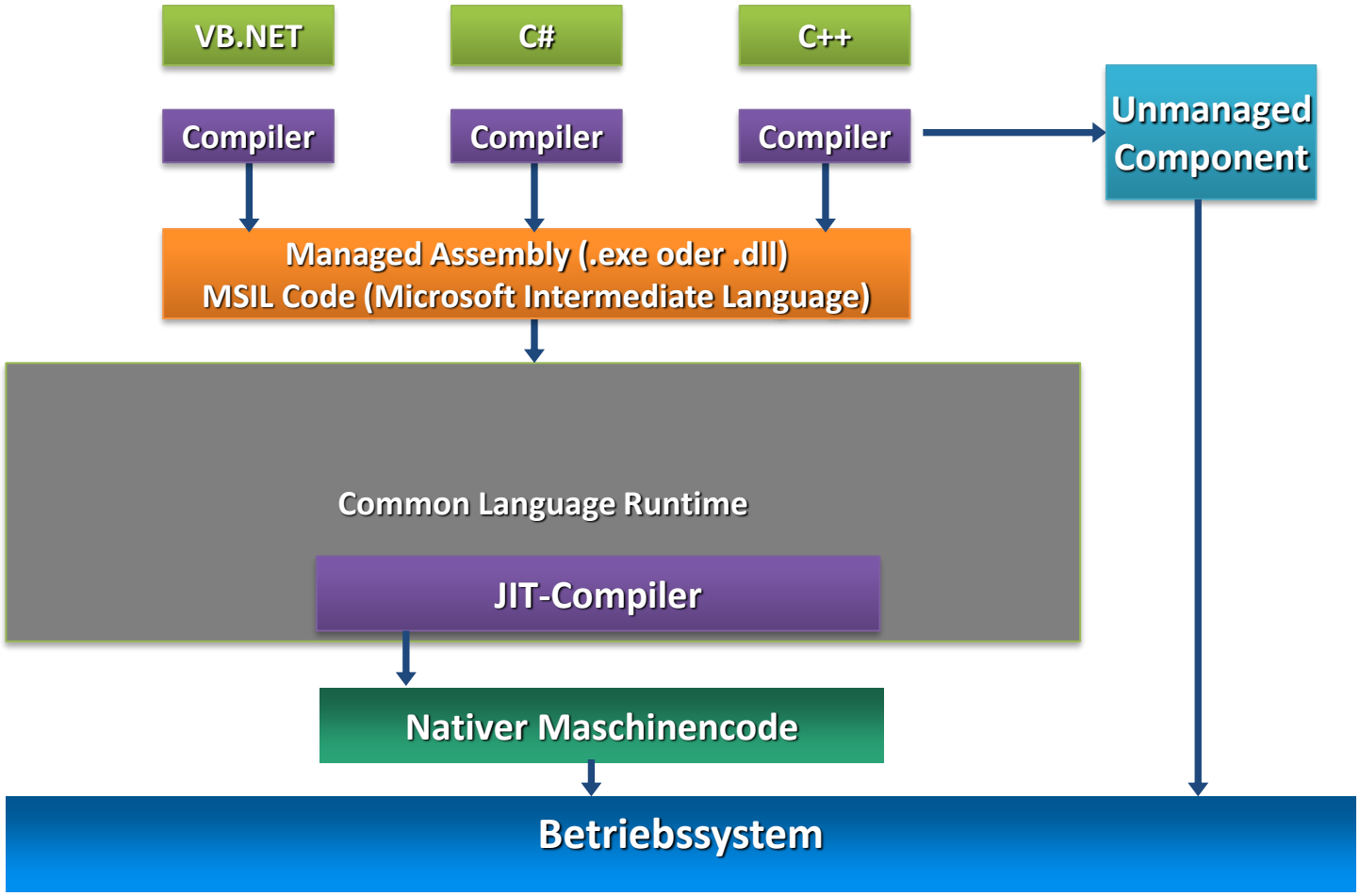
Managed Code

- Code wird unter Kontrolle der CLR ausgeführt
- Kein direkter Zugriff auf Rechnerressourcen
- Alle Sprachen

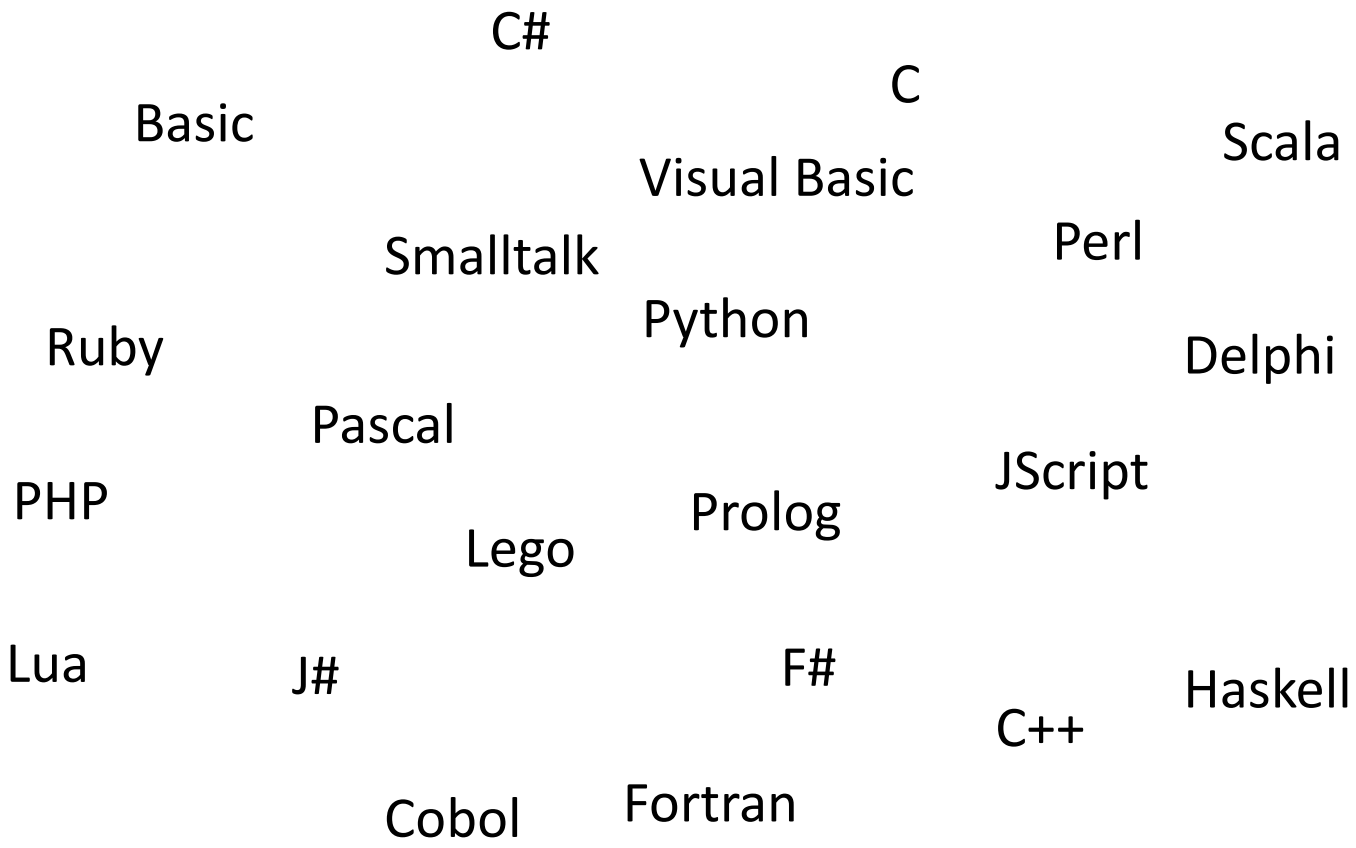
Unmanaged Code

- Es wird Code ausgeführt, der nicht unter der Kontrolle der CLR abläuft
- Direkter Zugriff auf Ressourcen
- C++ (und per spezieller Einstellung C#)

Ausführungsschritte von .NET Code



.NET Sprachen



Garbage Collector (GC)

- GC ist Teil der Common Language Runtime
- Kümmert sich um die Speicherverwaltung
- Er reserviert Speicher für neue Objekte, und löscht Speicher, wenn sie nicht mehr verwendet werden
- Die Freigabe des Speichers erfolgt nichtdeterministisch, z.B. wenn der freie Speicher knapp wird
- deterministische Freigabe ist durch das IDisposable-Pattern möglich

Namespace

- Eindeutiger Pfad eines Objektes (ähnlich der Windows-Pfadangaben)
- Normalerweise folgender Aufbau
  - <Firma>.<Produktname>.<Modul> usw.
- Vorhandene System-Namespaces
  - System.Text, System.Web, System.Linq …

Namespaces in Klassen

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Ilci.ConsoleDemo
{
  class Program
  {
    static void Main(string[] args)
    {
    }
  }
}
```

- Definition via Schlüsselwort „namespace“
- Im Beispiel hat die Klasse „Program“ den Bezeichner „Ilci.ConsoleDemo.Program“
- Abkürzung der Bezeichner durch „using“

Assembly

- Kompilierte Klassen werden in einer sogenannten Assembly zusammengefasst und bereitgestellt
- Eine Assembly kann in Form einer Dynamic Link Library (DLL) bzw. EXE vorhanden sein
- Verwendung via Projekt-Referenz bzw. bin-Ordner oder GAC
- Normalerweise kein Registry-Eintrag notwendig

Versionierung der Assembly

- Assemblies besitzen durch ein Attribut eine eindeutige Versionsnummer
- Projekt -> Properties -> `AssemblyInfo.cs` oder durch die Projekteigenschaften editierbar
- AssemblyVersion (wird von Framework benutzt) und AssemblyFileVersion (Versionsnummer in Datei-System)
- Struktur `<MajorVersion>.<MinorVersion>.<Build Number>.<Revision>`

```
[assembly: AssemblyVersion("1.0.0.0")]
[assembly: AssemblyFileVersion("1.0.0.0")]
```

.NET Core

Was ist .NET Core

- Plattformübergreifendes Framework mit dem Ziel so universell wie möglich eingesetzt zu werden
- Open Source
- Bibliotheken werden durch NuGet verteilt
- Typische Einsatzszenarien sind zurzeit ASP.NET Webanwendungen, Konsolen-Apps sowie UWP-Apps
- Ist keine Untermenge des .NET Framework
- Besitzt ein Command Line Interface (CLI)

Full .NET Framework vs. .NET Core

Full .NET Framework	.NET Core
Vollständiges „etablierte“ Framework	Modulare Version des .NET Framework
Nur Windows	Cross-Platform
	Keine Untermenge vom .NET Framework
	Windows, Linux, Mac
	Implementation des .NET Standard

Entscheidung zwischen Full .NET Framework vs. .NET Core

Full .NET Framework	.NET Core
Aktuelle Projekt(e) nutzen .NET Framework	Cross-Platform
Third-Party .NET Libraries	Docker Container
.NET Technologien, die nicht für .NET Core vorhanden sind	Side-by-Side .NET Versions in Applikationen

https://docs.microsoft.com/de-de/dotnet/articles/standard/choosing-core-framework-server

https://dotnet.microsoft.com/platform/dotnet-standard#versions

Was ist .NET Standard 2.0

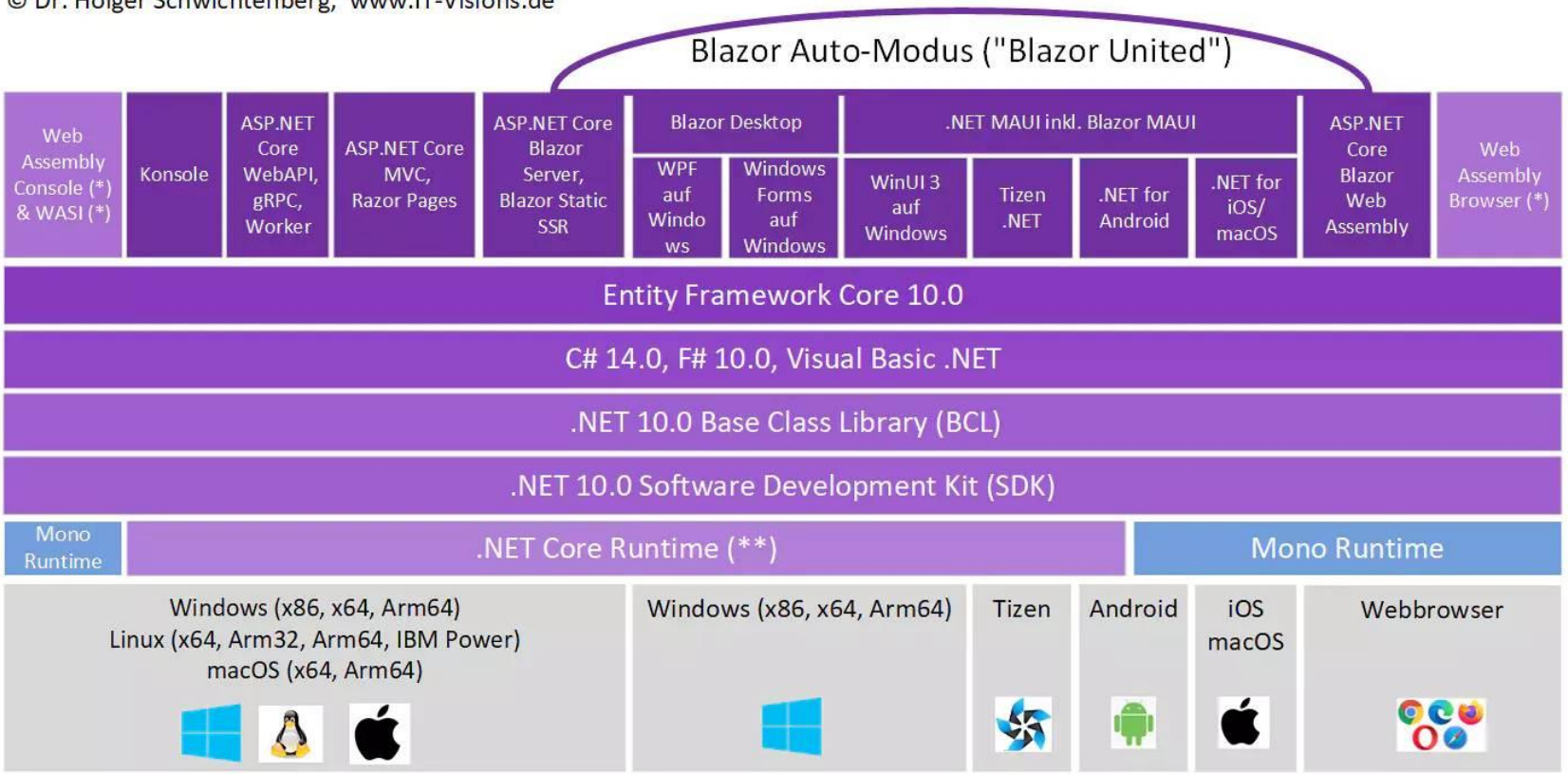
„Der Standard ist eine technische Vorgabe für Programmierschnittstellen (APIs), die es ermöglicht, dass sich Programmcode zwischen verschiedenen Implementierungen von .NET austauschen lässt. Entwickler, die eine Klassenbibliothek auf Basis einer Version des .NET Standard realisieren, können sicher sein, dass diese Bibliothek auf allen .NET-Implementierungen funktioniert, die diese Version des Standards umsetzen.“

- Immo Landwerth Programmanger .NET Entwicklungsteam

.NET Familie 2025/2026

Aufbau von .NET 10.0

© Dr. Holger Schwichtenberg, www.IT-Visions.de



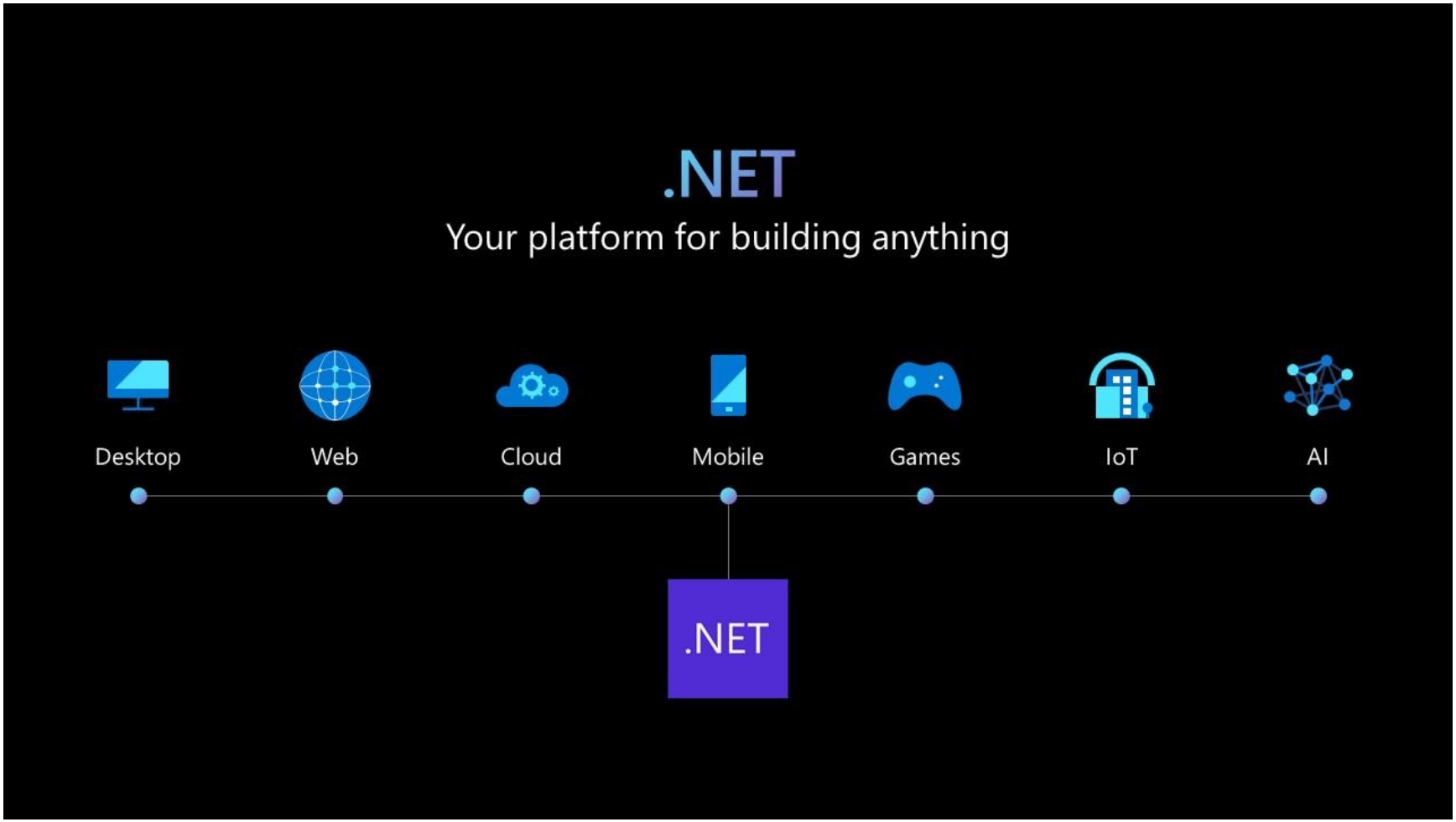
(*) Experimentelle Implementierungen ohne Support seit .NET 7.0 und auch noch in .NET 10.0

(**) .NET for Android auf .NET Core Runtime ist experimentell. Standard bei .NET for Android ist weiterhin die Mono Runtime.

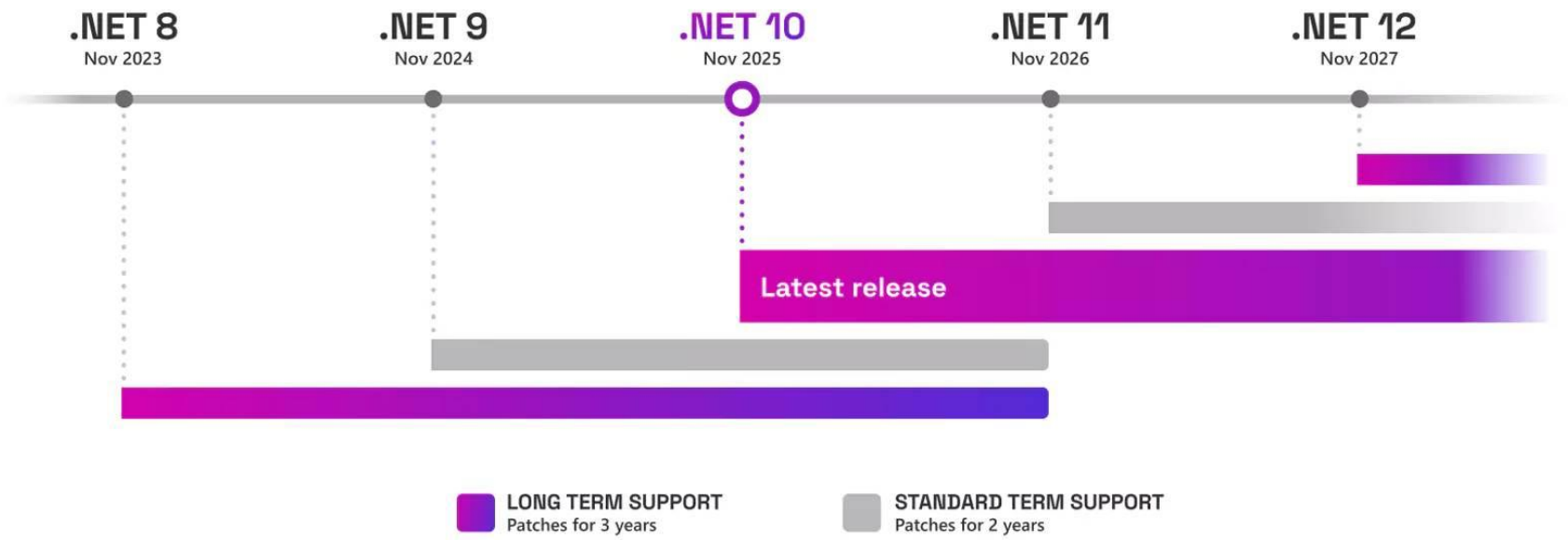
https://www.heise.de/news/NET-10-0-ist-fertig-11074367.html

Holger Schwichtenberg Stand: 12.November 2025

.NET 10 Platform



.NET Schedule



https://dotnet.microsoft.com/en-us/platform/support/policy

.NET CLI

- Command Line Application, die zum Entwickeln von Anwendungen auf allen Plattformen verwendet wird
- Führt die App aus und hosted die CLR
- OpenSource / SDK
- Erweiterbar (z.B. Entity Framework Befehle)
- Dokumentation
  - https://docs.microsoft.com/de-de/dotnet/core/tools/?tabs=netcore2x

.NET CLI Befehle „dotnet“ (Auszug)

Befehl	Beschreibung
new	Erstellt ein neues Projekt, eine Konfigurationsdatei oder eine Lösung auf Grundlage der angegebenen Vorlage.
restore	Stellt die Abhängigkeiten und Tools eines Projekts wieder her
build	Erstellt ein Projekt und alle seine Abhängigkeiten
publish	Packt die Anwendung und ihre Abhängigkeiten in einen Ordner für die Bereitstellung auf einem Hostsystem
run	Führt Quellcode ohne explizite Kompilierungs- oder Startbefehle aus.
test	.NET-Testtreiber, der verwendet wird, um Komponententests auszuführen.
pack	Packt den Code in ein NuGet-Paket
clean	Löscht die Ausgabe eines Projekts.
sln	Ändert eine .NET Core-Projektmappendatei.
store	Speichert die angegebenen Assemblys im Laufzeitpaketspeicher

Visual Studio

- Microsofts integrierte Entwicklungsumgebung (IDE)
- Vielzahl von Projekttypen
  - Windowsanwendungen, Web, Store, CrossPlatform, Cloud, Datenbank etc.
- Auszug unterstützte Programmiersprachen
  - C/C++,C#, Python, Html, CSS, JavaScript, TypeScript, VB.NET usw.
- Verschiedene Editionen

Visual Studio neues Projekt

Projektmappe

- Projektmappenverzeichnis anlegen
- Projektmappendatei (*.sln bzw. *.slnx) anlegen

Projekt

- Projektverzeichnis anlegen
- Projektdatei (*.csproj) anlegen

Assembly

- Verzeichnis für Eigenschaften einrichten
- Weitere Verzeichnisse z.B. für Ressourcen

Debug / Release

- Unterhalb des bin-Ordners 2 Verzeichnisse, jeweils für die Debug-Version und die Release-Version

Hands On



Hallo Welt +
Visual Studio Entwicklungsumgebung

Grundlagen C#

Generelle C# Spracheigenschaften

- Objektorientierte Programmiersprache
- Ecma-334 spezifiziert
- Case-Sensitiv d.h. Unterscheidung zwischen Groß- und Kleinschreibung
- Reservierte Schlüsselwörter
- Namensgebungsstandard ist Pascal Casing
  - Ausnahme: (Member) Variablen, Parameter

Grundlagen C#

- Variablen / Datentypen
  - Variablen / Konstanten
  - Primitive Datentypen
  - Zeichen & Zeichenketten
  - Implizite & explizite Konvertierung
  - Ausdrücke & Operatoren
  - Arrays & Enumerationen
  - Werte – und Referenztypen
- Codeblöcke & Kontrollstrukturen
  - Blöcke & Gültigkeitsbereiche
  - Bedingte Anweisungen
  - Verzweigungen
  - Schleifen
  - Iterationen

Variablen / Datentypen

- Eine Variable dient zur vorübergehenden Speicherung der Daten während der Laufzeit eines Programmes
- Sie besitzt einen eindeutigen Namen und einen Datentyp
- Für Namen von Variablen gelten folgende Regeln
  - Beginn mit einem Buchstaben
  - Buchstaben, Zahlen und einige wenige Sonderzeichen (z.B. Underscore) zulässig
  - Innerhalb eines Gültigkeitsbereichs darf es keine identische Variablennamen geben

Variablen / Datentypen

- Datentypen bestimmen den Typ und die Größe der Variablen
  - Ähneln verschiedenen Gefäßgrößen wie z.B. Gläsern








Variablen

Variablen

- Platzhalter für ‚Dinge', mit denen das Programm arbeitet und deren Inhalt ausgetauscht werden kann.

Definition

- Für eine Variable muss angegeben werden, von welchem Typ sie ist.
- Diese Definition kann an beliebiger Stelle im Code vorgenommen werden, muss aber vor der Verwendung der Variablen erfolgen

Zuweisung

- Einer Variablen kann einen Inhalt zugewiesen bekommen.
- Die Zuweisung kann auch schon bei der Deklaration vorgenommen werden (explizite Definition).
- Einer Variablen muß etwas zugewiesen werden, damit mit ihr weitergearbeitet werden kann.

Gültigkeit von Variablen

gültig

- eine Variable darf immer nur einmal innerhalb eines Blocks definiert werden
- eine Variable ist innerhalb eines Blocks und in allen untergeordneten Blöcken gültig, kann dort also verwendet werden
- bei einer for-Schleife gilt die Laufvariable als für den Block der Schleife definiert

nicht gültig

- in einem untergeordneten Block darf die Variable eines übergeordneten Blocks nicht noch einmal definiert werden (wohl aber darf sie verwendet werden)
- in einem übergeordneten Block darf eine Variable nicht noch einmal definiert werden, wenn sie zuvor in einem untergeordneten Block definiert wurde
- in einem übergeordneten Block kann nicht auf eine Variable zugegriffen werden, die in einem untergeordneten Block definiert wurde

Primitive Datentypen

Typ	.NET Typ	Beschreibung	Bit	Bereich
byte	Byte	Ganze Zahl ohne Vorzeichen	8	0 bis 255
sbyte	SByte	Ganze Zahl mit Vorzeichen	8	-128 bis 127
int	Int32	Ganze Zahl mit Vorzeichen	32	-2.147.483.648 bis 2.147.483.647
uint	UInt32	Ganze Zahl ohne Vorzeichen	32	0 bis 4294967295
short	Int16	Ganze Zahl mit Vorzeichen	16	-32.768 bis 32.767
ushort	UInt16	Ganze Zahl ohne Vorzeichen	16	0 bis 65535
long	Int64	Ganze Zahl mit Vorzeichen	64	-9223372036854775808 bis 9223372036854775807
ulong	UInt64	Ganze Zahl ohne Vorzeichen	64	0 bis 18446744073709551615
float	Single	Gleitkommatyp mit einfacher Genauigkeit	32	-3.402823e38 bis 3.402823e38 (Genauigkeit: 7 Stellen)

Primitive Datentypen

Typ	.NET Typ	Beschreibung	Bit	Bereich
double	Double	Gleitkommatyp mit doppelter Genauigkeit	64	-1.79769313486232e308 bis 1.79769313486232e308 (Genauigkeit: 15-16 Stellen)
char	Char	Ein einzelnes Unicode-Zeichen	16	Im Text verwendete Unicode-Symbole
bool	Boolean	Logischer boolescher Typ	8	True oder False
decimal	Decimal	Präziser Bruchtyp oder ganzzahliger Typ, der Dezimalzahlen mit 29 signifikanten Stellen darstellen kann	128	±1,0 × 10e−28 zu ±7,9 × 10e28 (Genauigkeit: 28-29 Stellen)

Implizite Typkonvertierungen

- Konvertierung (casting) von kleineren zu größeren Datentypen bzw. allgemeineren Datentypen

von	nach	nach	nach	nach	nach
byte	short	int	long	float	double
short	int	long	float	double	
int	long	float	double		
long	float	double			
float	double				

Explizite Typkonvertierung

- Gefahr der Inkonsistenz bei explizierter Konvertierung von größerem Datentyp in kleineren

```
double pi = 3.14159265;
int converted = (int) pi; //Ausgabe => 3
```

- Inkompatible Datentypen können nicht konvertiert werden

```
string text = "Dies ist ein Test";
int neuerText = (int) text; //Fehler
```

- Explizite Konvertierung via Convert-Klasse

Datentypen Reloaded

- Zeichen & Zeichenketten
  - **`char`**
  - **`string`**
- Logischer Typ
  - **`bool bzw. boolean`**
- Sonderfall **`var`** und **`dynamic`**

Char

- Ein char (16 Bit) hat 2 verschiedene Repräsentationsarten
  - als *Zeichen* wie z.B 'a'
  - als *Zahl* z.B. 97, dass ein kleines 'a' repräsentiert
- Verwendung mit Hochkommata
- Kleinster Teil eines Strings

```
string text = "Dies ist ein Kurs";
char zeichen = text[2];          // Buchstabe ‚e‘
byte zahl = (byte) zeichen;      // Zahl 101
```

String

- Strings sind Zeichenketten in Unicode
  - Äquivalent zu einem char-Array
  - Einzelne Zeichen können via [] ermittelt werden
  - Immutabel
  - Hilfsmethoden
    - Length
    - ToUpper
    - ToLower
    - IndexOf
    - Substring
    - …

Boolean

- Wahrheitswert true/false
- Logische Ausdrücke
  - and, or, xor
- Diverse Operatoren
  - ! => not
  - | => or
  - & => and
  - ^ => xor

Sonderfall var & dynamic

- **`var`** ist eine *implizit typisierte lokale* Variable, d.h. der Typ wird vom Compiler bestimmt und muss bei Deklaration gesetzt werden

```
var i = 5; //Typ int
var s = "Hallo"; //Typ string
```

- **`dynamic`** ist eine *vorerst untypisierte* Variable während der Kompilierzeit, d.h. der Typ wird beim kompilieren nicht überprüft. Dies wird stattdessen während Laufzeit getan
  - Anwendbar als Typ in Eigenschaft,Feld, Indexer, Parameter, Rückgabewerts oder lokalen Variable

```
dynamic dyn = 5;
dyn = dyn + 10.56; //Ausgabe -> 15, Typ double
```

Operatoren

- Arithmetische Operatoren
- Zuweisungsoperatoren
- Vergleichsoperatoren
- Logische Operatoren

Arithmetische Operatoren

+	• Addition von zwei numerischen Werttypen • Positives Vorzeichen vor einem numerischen Werttyp
-	• Subtraktion von zwei numerischen Werttypen • Negatives Vorzeichen vor einem numerischen Werttyp
*	• Multiplikation von zwei numerischen Werttypen
/	• Division von zwei numerischen Werttypen
%	• Modulo => Restwert einer Division zweier Operanden
++	• Inkrement eines Werttyps als Präfix (+1 vor Auswertung des Ausdrucks) • Inkrement eines Werttyps als Suffix (+1 nach Auswertung des Ausdrucks)
--	• Dekrement eines Werttyps als Präfix (-1 vor Auswertung des Ausdrucks) • Dekrement eines Werttyps als Präfix (-1 vor Auswertung des Ausdrucks)

Zuweisungsoperatoren

=	• Weist x den Wert von y zu
+=	• Weist x den Wert von x + y zu
-=	• Weist x den Wert von x - y zu
/=	• Weist x den Wert von x / y zu
*=	• Weist x den Wert von x * y zu
%=	• Weist x den Wert von x % y zu

Vergleichsoperatoren

==	• Gleichheitsoperator
!=	• Ungleichheitsoperator
<	• Kleiner-als-Operator
>	• Größer-als-Operator
<=	• Kleiner-oder-gleich-Operator
>=	• Größer-oder-gleich-Operator

Logische Operatoren

!	• Logischer Umkehrung (Invertierung)
&&	• Logischer UND-Operator (kurzschlüssige Auswertung)
||	• Logischer ODER-Operator (kurzschlüssige Auswertung)

Hands On



Typen & Konvertierung & Operatoren

Arrays / Enums

Array

- Ein Array ist ein Behälter für mehrere Objekte mit einer fixen Größe
  - z.B. int[] lottozahlen = new int[6]
- Zugriff auf die Elemente über Index (skalarer Wert)
  - Array ist 0-Index-basiert => lottozahlen[0]
  - Letztes Element lottozahlen[Anzahl Elemente -1]
- Kann nach Erstellung weder vergrößert noch verkleinert werden

Enums

- Ein Enum ist ein Aufzählungstyp
- Einzelne Aufzählungswerte haben standardmäßig einen Integer-Wert zu ihren Namen zugeordnet
  - Fortlaufende Nummerierung bzw. explizite Wertzuweisung

```
enum Wochentag {
 Montag = 1,
 Dienstag,
 Mittwoch
 …
};

Wochentag wt = Wochentag.Dienstag;
int wochentag = (int) wt;
```

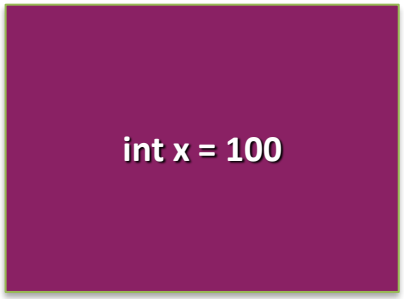
Hands On

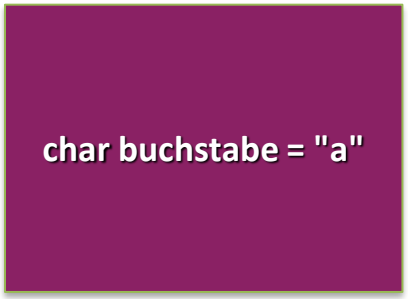


Array, Enum

Werttypen

- Auf Werttypen (Value Types) basierende Variablen enthalten die tatsächlichen Werte
  - Beim Kopieren wird der enthaltene Wert der Variablen kopiert
- Viele primitive Datentypen sind Werttypen
  - Z.B. char, int, float, double





Übergabe als Wert vs. Referenz

Wert

- Variable muss vor Übergabe definiert (nicht nur deklariert) werden
- Wert kann in der aufgerufenen Methode beliebig verändert werden
- Nach Rücksprung bleibt der ursprüngliche Wert unverändert erhalten

ref

- Variable muss vor Übergabe definiert (nicht nur deklariert) werden
- Veränderungen in der aufgerufenen Methode führen zu Veränderungen der ursprünglichen Variablenwerte
- sowohl der Parameter als auch der Aufruf müssen mit dem Schlüsselwort **ref** gekennzeichnet werden

out

- Variable muss vor Übergabe nur deklariert (nicht auch definiert) werden
- Veränderungen in der aufgerufenen Methode führen zu Veränderungen der ursprünglichen Variablenwerte
- sowohl der Parameter als auch der Aufruf müssen mit dem Schlüsselwort **out** gekennzeichnet werden

Hands On



ValueTypen / ReferenzTypes

Kontrollstrukturen

- Bedingte Anweisungen
- Verzweigungen
- Schleifen
- Iterationen
- Schleifen-Steuerung

If/else – bedingte Anweisungen

- Logischer Ausdruck bzgl. Wahrheitsüberprüfung
  - True/False – Statement
- Kann verschachtelt werden
- Vereinfachung durch ?: - Operator

```
if(name == "Bill Gates") //wenn wahr
{
   gehalt *= 1000;
}
else //ansonsten wenn falsch
{
   gehalt /= 1000;
}

//gehalt = (name == "Bill Gates") ? gehalt * 1000 : gehalt/1000;
```

Hands On



IfElse

Verzweigungen mit switch

- Kurzversion von if-else-Bedingung
- Vergleich auf vorgebene Festwerte
- Möglichkeit eine Standardauswahl zu definieren (default)

```
Console.WriteLine("Bitte geben Sie eine Automarke ein")
string autoMarke = Console.Readline();

switch(autoMarke)
{
  case "Mercedes":
        Console.WriteLine("Das Beste oder nichts"); break;
  case "VW":
        Console.WriteLine("Ab zur Abgasuntersuchung"); break;
  default:
        Console.WriteLine("08/15 ist auch nicht schlecht!");break;
}
```

Switch - Pattern Matching

- Pattern Matching prüft Werte nicht nur auf feste Konstanten
- Mögliche Muster: Typ, Konstante, Eigenschaften, Bereichs- und Logikmuster
- Switch-Ausdruck liefert direkt einen Wert zurück
- Kombinierbar mit when-Klauseln und Null-Prüfungen
- Besonders nützlich bei Records, Hierarchien und komplexen Bedingungen

```
public static string Beschreibe(object obj) => obj switch
{
  null => "Kein Wert",
  Kunde { Kundennummer: > 1000 } k => $"Großkunde {k.Vorname}",
  Kunde k => $"Kunde {k.Vorname}",
  Person { Vorname: var v, Nachname: "Meier" } => $"Meier: {v}",
  int n when n < 0 => "Negative Zahl",
  int n => $"Zahl {n}",
  _ => "Unbekannter Typ"
};
```

Hands On



Switch

Schleife mit for & Sprunganweisungen

- Klassische Schleife mit Initialisierabschnitt, Bedingung (logischer Ausdruck) und Iteration gefolgt vom Anweisungsblock
  - for(initialisierer;bedingung;interator)
    { Anweisungsblock }

```
for(int i = 0; i <= 10; i++)
{
  Console.WriteLine(i);
}
```

Iterationen mit foreach

- Alternative Schreibweise der for-Schleife
- Einfacher Durchlauf für Auflistungen
- Besteht aus Schleifenvariable und durchlaufbaren Liste

```
int[] lottozahlen = new int[] { 6,17,26,38,42,45 };
foreach(int zahl in lottozahlen)
{
   Console.WriteLine("Folgende Zahl wurde gezogen:" + zahl);
}
```

Schleifen mit while / do..while

- **`do..while`** wird so lange ausgeführt (mind. 1x) bis ein bestimmter Ausdruck **`false`** liefert

```
int i = 0;
do
{
  Console.WriteLine(i); i++;
}while(i < 5);
```

- **`while`** wird so lange ausgeführt (mind. 1x) bis ein bestimmter Ausdruck **false** liefert

```
int i = 0;
while(i < 5)
{
  Console.WriteLine(i); i++;
}
```

Schleifen-Steuerung

Anweisung	Beschreibung	Beispiel
break	Beendet die direkt umschließende Schleife	`for (int i = 1; i <= 100; i++) { if (i == 5) { break; } Console.WriteLine(i); }`
continue	Überspringt den aktuellen und geht zum nächsten Durchlauf der umschließenden Schleife	`for (int i = 1; i <= 100; i++) { if (i == 5) { continue; } Console.WriteLine(i); }`
goto	Springt direkt zur Anweisung mit Marke	`for (int i = 1; i <= 100; i++) { if (i == 5) { goto Ende; } Console.WriteLine(i); } Ende: Console.WriteLine("Ende");`

Hands On



Schleifen

Aufgaben

- Namen ausgeben
- Eingabe Rückwärts ausgeben
- 3 Eingaben Größte Zahl
- Wieviel Tage Lebst Du Schon?
- Primzahlen
- BubbleSort
- QuickSort
- Zufallszahl raten
- Euro – Umrechner
- Lottozahlen
- Taschenrechner

Objektorientierte Programmierung

- Ziele von Softwareentwicklung
- Klassen und Objekte
  - Klassen & Objekte
  - Methoden
  - Eigenschaften
  - Zugriffsmodifizierer
  - Konstruktoren
  - Statische Elemente
  - Vererbung / Abstraktion

Ziele von Softwareentwicklung

Anwenderperspektive

(„unproblematische Problemlösungen“)

- Korrektheit
- Fehlerfreiheit
- Benutzerfreundlichkeit

Entwicklerperspektive

(„bezahlbare Herstellung“

- Wartbarkeit
- Erweiterbarkeit
- Anpassbarkeit
- Redundanzvermeidung, Wiederverwendung

Reale Welt, Abstraktion und Programm

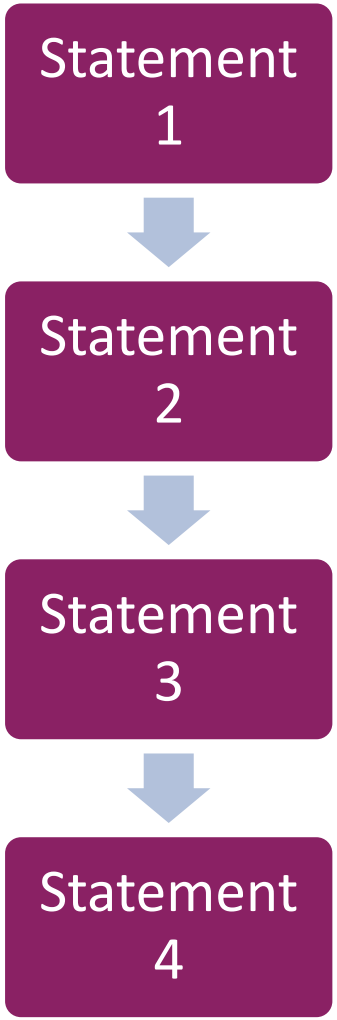


- Ein **Softwareprogramm** führt Arbeitsabläufe oder allgemeine Abläufe der realen Welt aus
- Ein **Model** beschreibt Elemente und Abläufe der realen Welt (Darstellung in UML)

Komplexitätsproblem

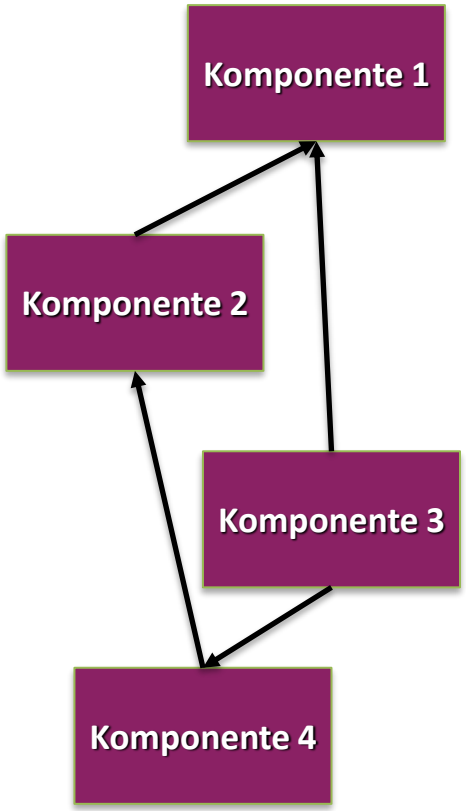
- Software ist inhärent komplex
- Steigerung auf
  - Anwenderseite
    (mehr Funktionen, mehr Bereiche)
  - Entwicklungsseite
    (technische Weiterentwicklungen,
    Systemumgebungen)

Imperative Programmierung



- Top-Bottom
- Statement-Abarbeitung
- Programmstruktur-Defizit
  (Neigung zu Spaghetti-Code)
- Weiterentwicklung zu prozeduraler Programmierung

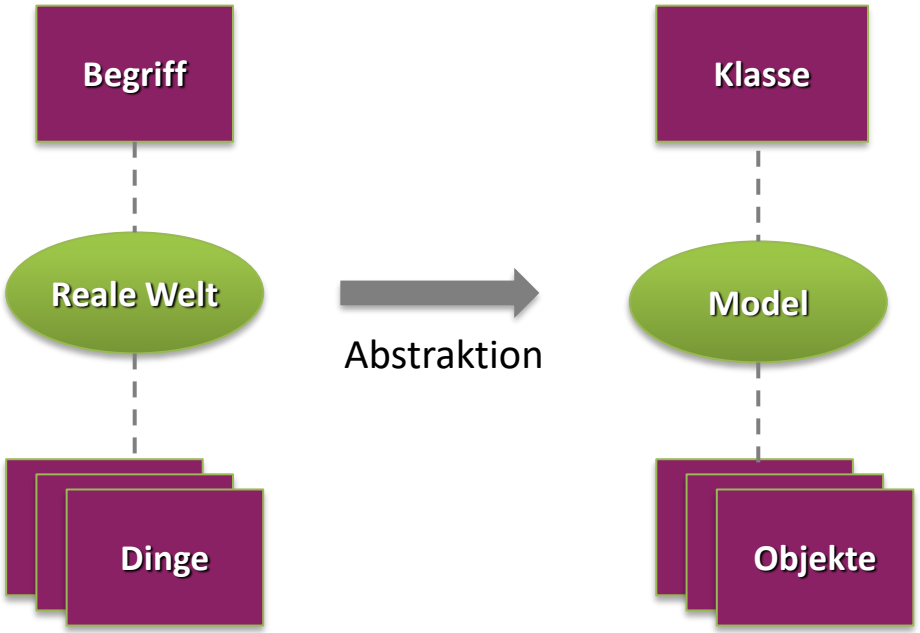
Komponenten/Modul Programmierung



- Kapselung von Programmfunktionen
- Einheitliches Interface
- Lose Kopplung von Modulen

Klassen und Objekte

- Eine Klasse ist ein Bauplan/Vorlage für ein Objekt und beschreibt den Aufbau gleichartiger Objekte
- Ein Objekt ist eine konkret umgesetzte „Instanz“ einer Vorlage bzw. Klasse



Klassen

- Klassen erzeugen mittels **new** –Operator Objekte, die über Referenzen angesprochen werden
- Klassen bzw. Objekte können Verbindungen (Beziehungen) zu anderen Klassen / Objekte haben
  - Assoziation
  - Vererbung
- Klassen haben Eigenschaften und Methoden (Aktionen) und Zugriffsmodifizierer

Beziehungen



Bank

Konto

Instance of

einKonto

Klasse, Bauplan,
Begriff

Objekt
konkretes Ding
(instance = Beispiel)

Referenz



Kunde

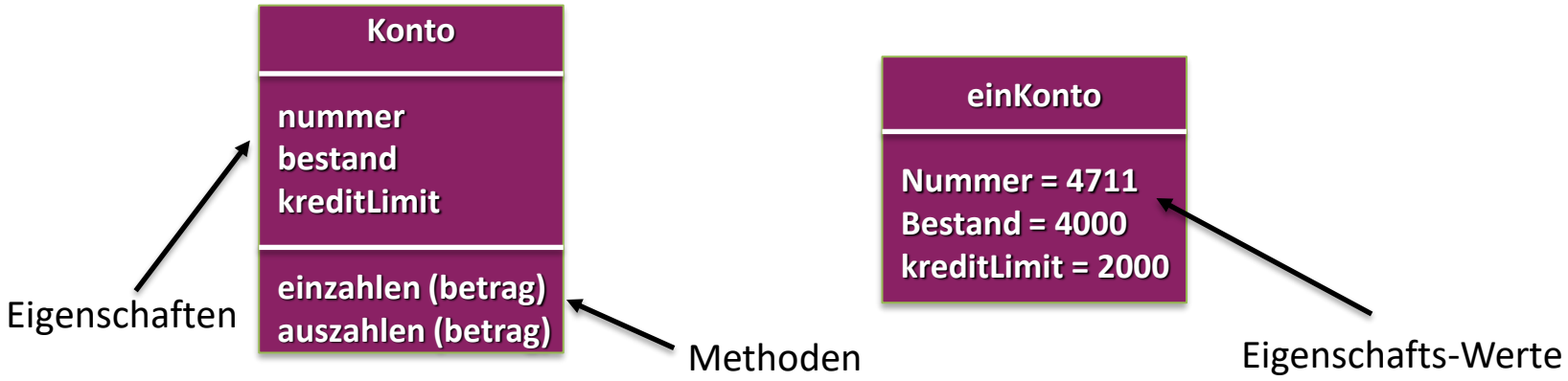
Ein Klasse kann
Viele Objekte erzeugen

k1 : Konto

k2 : Konto

Eigenschaften & Methoden

- Eigenschaften sind Variablen/Attribute eines Objekts
- Methoden sind Aktionen eines Objekts (Die -„Was kann ein Objekt?“-Frage)



Klassen in C#

```
public class Kunde
{
    //Verkürzte-Eigenschaft
    public string Name {get; set;}

    //Klassenmember/Attribut
    private string _vorname;

    private Konto _konto;

    //Eigenschaft
    public string Vorname
    {
      get { return _vorname; }
      set { _vorname = value; }
    }

    public void DoAnything()
    {}
}
```

- Schlüsselwort **class**
- Zugriffsmodifizierer
  - public,internal, protected, private
- Elemente
  - Variablen
  - (statische) Eigenschaften (Properties)
  - Methoden
  - Enumerations
  - Delegates
  - Events
  - Andere Klassen/Objekte

Datei – und Klassenorganisation

mehrere Klassen pro Datei

- verschiedene Klassen werden in einer Datei implementiert
- sie müssen dafür in einer Ebene liegen

eine Klasse in einer Datei

- Eine Klasse wird in einer Datei untergebracht

eine Klasse über mehrere Dateien

- Der Code für eine Klasse kann über mehrere Dateien verteilt werden
- Voraussetzung ist die Verwendung des Schlüsselwortes **partial**

(verschachtelte Klassen)

- innerhalb einer Klasse wird mit dem Schlüsselwort class eine weitere Klasse definiert
- die zweite Klasse ist dann nur noch über den Umweg der ersten Klasse erreichbar

zu beachten: Klassenbezeichnung und Dateiname müssen nicht übereinstimmen!

Records

- Records sind Referenztypen für überwiegend unveränderliche Daten
- Compiler generiert u. a. Equality, GetHashCode und ToString
- Value Equality: Zwei Records sind gleich, wenn alle Werte gleich sind
- Ideal für DTOs, API-Modelle, Wertobjekte und Ergebnisdaten
- Nicht ideal, wenn Identität, Verhalten oder Veränderlichkeit im Fokus stehen

```
public record Kunde(string Vorname, string Nachname, int Alter);

var k1 = new Kunde("Max", "Müller", 30);
var k2 = k1 with { Alter = 31 };

Console.WriteLine(k1 == new Kunde("Max", "Müller", 30)); // true
Console.WriteLine(k2); // Kunde { Vorname = Max, Nachname = Müller, Alter = 31 }
```

Records - Vererbung

- Records können voneinander erben: record Kunde : Person
- Die Basisklasse kann ebenfalls ein Record sein
- Value Equality berücksichtigt alle Eigenschaften der gesamten Vererbungskette
- Positionale Syntax: Basis-Record zuerst initialisieren, dann eigene Parameter
- with kann auch Eigenschaften der Basisklasse ändern
- sealed verhindert weitere Ableitung

```
public abstract record Person(string Vorname, string Nachname);

public record Kunde(string Vorname, string Nachname, int Kundennummer)
   : Person(Vorname, Nachname);

var k1 = new Kunde("Max", "Müller", 4711);
var k2 = k1 with { Nachname = "Meier" };

Console.WriteLine(k1 == new Kunde("Max", "Müller", 4711)); // true
Console.WriteLine(k2); // Kunde { Vorname = Max, Nachname = Meier,
Kundennummer = 4711 }
```

Zugriffsmodifikation

public	• Öffentlicher Zugriff (auch außerhalb der Klasse)
private	• Zugriff nur innerhalb der Klasse
protected	• Zugriff nur innerhalb der Klasse und abgeleiteten Klassen
internal	• Zugriff nur innerhalb der gleichen Assembly (Projekts)
protected internal	• Kombination aus protected und internal (Sichtbarkeit mit *oder* verknüpft)
sealed	• Von dieser Klasse kann nicht geerbt werden

Klassen vs. Statische Klassen

- Klassen können komplett als **`static`** deklariert werden und existieren nur einmal im Speicher
- Von statischen Klassen können keine Instanzen gebildet werden
- Deshalb müssen alle Member einer statischen Klasse ebenfalls statisch sein – auch die privaten Variablen
- Eigenschaften und Methoden von **nicht** statischen Klassen können mit **`static`** deklariert werden
  - Es sind dann private Variablen der Klasse und nicht der Objekte
  - Zugriff über Klassennamen
- Standardsichtbarkeit ist internal (alternative public)

Hands On



(Statische) Klassen

Konstruktoren

- Ein Konstruktor ist eine Spezial-Methode, die nur einmal beim Erzeugen des Objekts ausgeführt wird
- Über diese Methode initialisiert sich das Objekt am Anfang mit sinnvollen Daten
- Ist kein Konstruktor explizit eingerichtet, wirkt ‚im Hintergrund' automatisch ein parameterloser Konstruktor
- Konstruktoren haben keinen Rückgabetyp, nicht einmal void
- Ein Objekt kann nur mit definierten Konstruktoren instanziiert werden (oder mit (), wenn kein Konstruktor implementiert wurde)
- Sind parametrisierte Konstruktoren eingerichtet, muß ein parameterloser Konstruktor explizit eingerichtet werden, wenn weiterhin eine Instanziierung mit () möglich bleiben soll
- Konstruktoren können verkettet werden

Methoden

```
[Modifizierer] Typ Bezeichner([ParameterListe])
{
  …
  return Wert
}
```

- Methoden können, müssen aber nicht Rückgaben haben (void)
- Standardsichtbarkeit ist **`private`**
- Aufruf mit Objektreferenz.Methode bei nicht statischen Methoden und Klassenname.Methode bei statischen
- Können überladen (verschiedene Signaturen) bzw. überschrieben werden

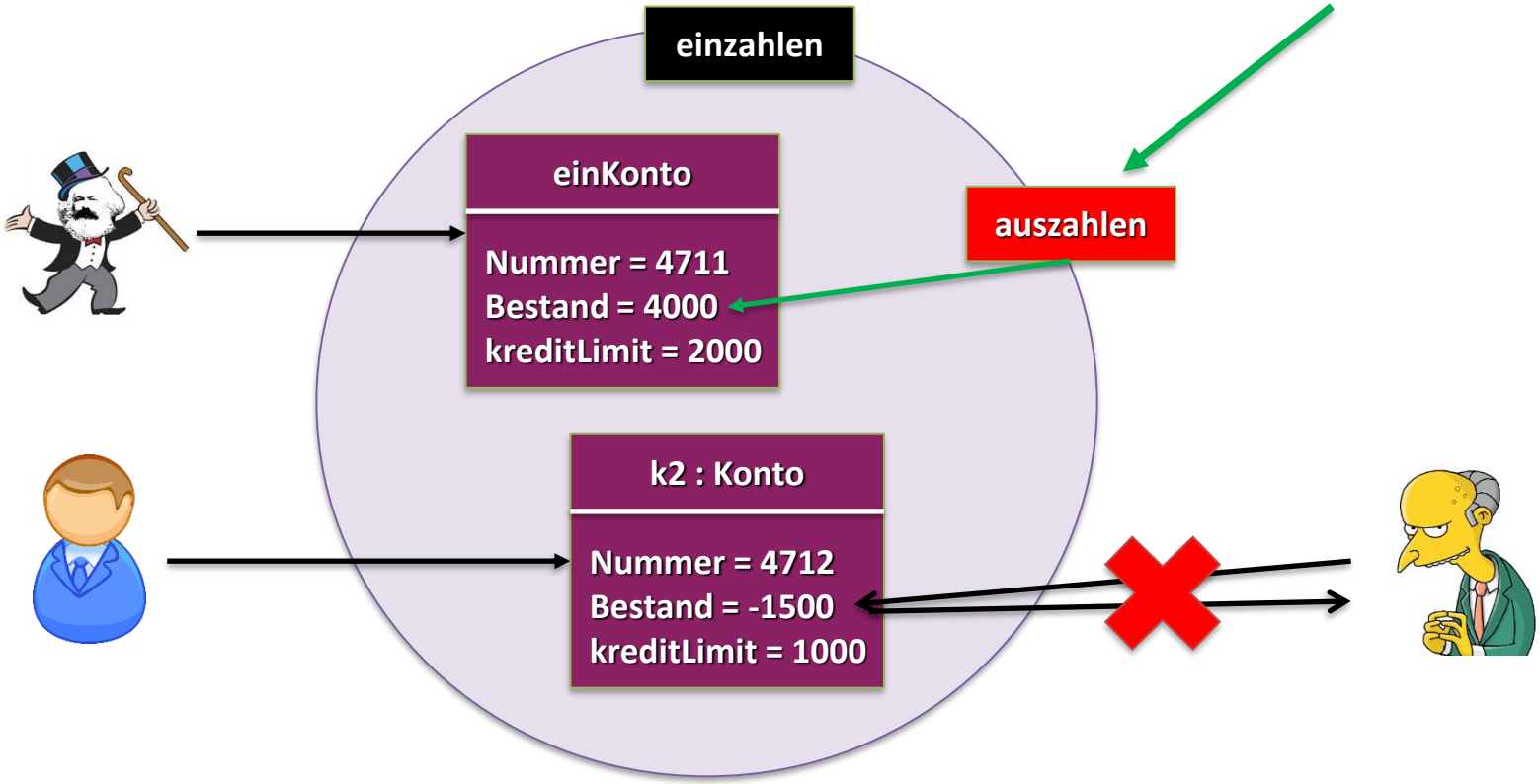
Methodenüberladung

- Verschiedene Versionen von Basisfunktionalitäten
- Bedingungen von gültiger Methodenüberladung
  - Unterschiedliche Anzahl von Parametern
  - Bei gleicher Parameteranzahl mindestens ein unterschiedlicher Datentyp

```
public void DoSomething();
public void DoSomething(byte x);
public void DoSomething(int x);
public void DoSomething(int x , string name);
```

Datenkapselung

- Ein Objekt übernimmt die Verantwortung für die Konsistenz seiner Daten und für das Resultat seiner Arbeit/Aufgaben



Getter / Setter

Variable als private kennzeichnen, so daß sie von außen ohne weiteres nicht mehr zugänglich ist

Entsprechenden Getter/Setter implementieren, der übergebene Werte entgegennimmt und in den Membervariablen speichert

Die Klasse hat die vollständige Kontrolle über die Zuweisung von Eigenschaften

Verantwortung der Konsistenz

Getter / Setter Varianten

nur lesen

- eine Eigenschaft wird zu einer nur lesenden Eigenschaft, indem nur der Getter auf public gesetzt wird
- Ist der Setter nicht öffentlich, kann auch nichts in die Eigenschaft geschrieben werden

nur schreiben

- In diesem Fall wird nur der Setter öffentlich gemacht, die Eigenschaft kann also gesetzt werden
- Der Getter wird nicht öffentlich gemacht, der Wert der Eigenschaft kann also nicht ausgelesen werden

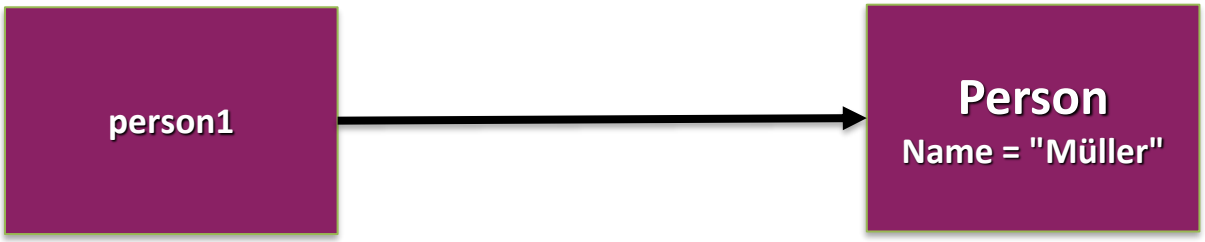
gar nicht zugänglich

- werden beide, der Getter und der Setter nicht öffentlich gemacht, kann von außen die Eigenschaft weder gesetzt noch ausgelesen werden

C# 6 Autoproperties mit nur Getter und Initializer

Referenztypen

- Variablen von Referenztypen (reference type) speichern Verweise auf ihre Daten (Objekte)
- Klassen und Objekte sind Referenztypen
- Default-Wert für Referenzentypen ist „**`null`**“
- Multiple Variablen können auf das gleiche Objekt verweisen
- Objekte werden im Speicher durch **`new`** allokiert



Werttypen vs. Referenztypen

Werttypen

- Variable enthält Wert
- gespeichert auf dem Stack
- wird mit Werten initialisiert

Referenztypen

- Variable enthält Referenz (Zeiger) auf ein Objekt
- gespeichert im Heap
- wird mit null initialisiert

Konsequenzen bei Referenztypen

- Bei einer Zuweisung (z.B. ‚k1 = k2‘) wird die Referenz zugewiesen (und nicht etwa ein Duplikat des Objekts). Beide Variablen zeigen auf dasselbe Objekt und prozessieren dasselbe Objekt.

- Bei einem Vergleich z.B. wird nicht der Inhalt (Eigenschaften) von Objekten verglichen, sondern, ob gleiche Referenzen vorliegen (also beide Variablen auf dasselbe Objekt zeigen).

Null

- Ein Objekt kann auch mit einer null-Referenz definiert werden (ein null-Zeiger ist auch ein Zeiger und damit ‚mehr als Nichts‘).
- Eine Variable kann jederzeit zurückgesetzt werden, erhält dabei dann einen null-Zeiger.
- Eine Variable kann daraufhin geprüft werden, ob sie einen Nullzeiger enthält.

Hands On



OOP

Vererbung

- Eine Klasse kann von einer anderen Klasse erben
  - Durch „**:**“ getrennt wird angegeben von welcher Klasse geerbt wird
  - Es kann immer nur von ***einer*** Klasse geerbt werden
- Alle Klassen/Objekte erben implizit von **`object`**

```
public class Fahrzeug
{}

public class Pkw : Fahrzeug //Pkw ist ein Fahrzeug
{}

public class Lkw : Fahrzeug
{}
```

Kontruktorenverkettung base/this




```
public class Fahrzeug
{
   private string _kennzeichen;
   public Fahrzeug(string kennzeichen)
   {
      _kennzeichen = kennzeichen;
   }
}

public class Pkw : Fahrzeug
{
   public Pkw(string kennzeichen,string typ) : base(kennzeichen){}

   publick Pkw(string kennzeichen) : this(kennzeichen,string.Empty)
}
```

Überschreiben

virtual

- ermöglicht das Überschreiben einer Methode oder Eigenschaft

override

- überschreibt die Methode/Eigenschaft einer Basisklasse

Überschreiben

```
public class Fahrzeug
{
   private string _kennzeichen;
   public Fahrzeug(string kennzeichen)
   {
      _kennzeichen = kennzeichen;
   }

   public virtual void Fahren()
   {…}
}

public class Pkw : Fahrzeug
{
  …
  public override void Fahren()
  {  … }
}
```

Abstrakte Klassen / Methoden / Eigenschaften

- ist auch nur eine Eigenschaft oder eine Methode abstrakt, **muss** die ***gesamte*** Klasse abstrakt sein
- abstrakte Methoden/Eigenschaften dürfen keinen Code enthalten
- ist die Klasse abstrakt, kann kein Objekt mehr mit ihr instanziiert werden
- eine abstrakte Klasse kann nach wie vor Methoden/Eigenschaften implementieren
- abstrakte Methoden/Eigenschaften müssen von den abgeleiteten Klassen mit **override** implementiert werden

Abstrakte Klassen

```
public abstract class Fahrzeug
{
   private string _kennzeichen;
   public abstract int Achsen {get; set;}

   public abstract void Fahren()
   {…}
}

public class Pkw : Fahrzeug
{
  …
  public override void Fahren()
  {  … }

  public override int Achsen
  {
     get { return 4; }
  }
}
```

Hands On



Vererbung

Interfaces

Was ist eine Schnittstelle?

*„Eine **Schnittstelle** (englisch interface) dient in der objektorientierten Programmierung der Vereinbarung gemeinsamer Signaturen von Methoden, die in unterschiedlichen Klassen implementiert werden. Die Schnittstelle gibt dabei an, welche Methoden vorhanden sind oder vorhanden sein müssen.“*

Quelle: Wikipedia

Regeln für Schnittstellen

- Schnittstellen sind immer public
- Sie beginnen per Konvention immer mit einem vorangestelltem „**I**“
- Alle Eigenschaften und Methoden von Schnittstellen sind immer public (deshalb gibt es keine Zugriffsmodifizierer)
- Eine Klasse kann beliebig viele Schnittstellen implementieren
- Eine erbende Klasse erbt die Verpflichtung alle Member des Interfaces zu implementieren
- Implementiert eine abgeleitete Klasse Schnittstellen, wird nach dem ‚**:**‘ erst die Basisklasse, dann die Schnittstellen angegeben (z.B. „KlasseB : KlasseA, ISchnittstelle1, ISchnittstelle2“)
- eine Schnittstelle kann andere Schnittstellen implementieren

Implementierung von Schnittstellen

```
interface IFahrzeug
{
  string Kennzeichen { get; set; }
  string Typ { get; set; }
  void Fahren();
}
```

```
class Pkw : IFahrzeug
{
   private string _kennzeichen;

   public string Kennzeichen
   {
      get { return _kennzeichen; }
      set { _kennzeichen = value; }
   }

   public void Fahren()
   {
     Console.WriteLine("Brrrmmmmmm…");
   }
}
```

```
class Lkw : IFahrzeug
{
   private string _kennzeichen;

   public string Kennzeichen
   {
      get { return _kennzeichen; }
      set { _kennzeichen = value; }
   }

   public void Fahren()
   {
     Console.WriteLine("LKW Brrrmmm…");
   }
}
```

Schnittstelle vs. Abstrakte Klassen

Schnittstelle

- Keine Implementierungen möglich (Code)
- Alle Member sind implizit **public**
- Keine internen Variablen
- Mehrfachimplementierung möglich

Abstrakte Klasse

- Kann Implementierung (Code) enthalten
- Unterschiedliche Sichtbarkeitsstufen möglich
- Interne Variablen möglich
- Nur Einfachvererbung möglich

Hands On



Interfaces +
Aufgabe Fahrzeuge

Ausnahmebehandlung

Themen

- Was sind Exceptions?
- Ausnahmebehandlung
- Exception-Typen und Hierarchie
- Best Practices

Was sind Exceptions?

- Durch Programmierfehler und/oder fehlerhafte Umstände (z.B. irreguläre Eingabedaten, unterbrochene Netzwerkverbindungen) kann die reguläre Ausführung von Code scheitern (Laufzeitfehler)

- Ziel ist die angemessene Meldung und Behandlung von diesen Ausnahmen ohne Absturz bzw. Datenverlust

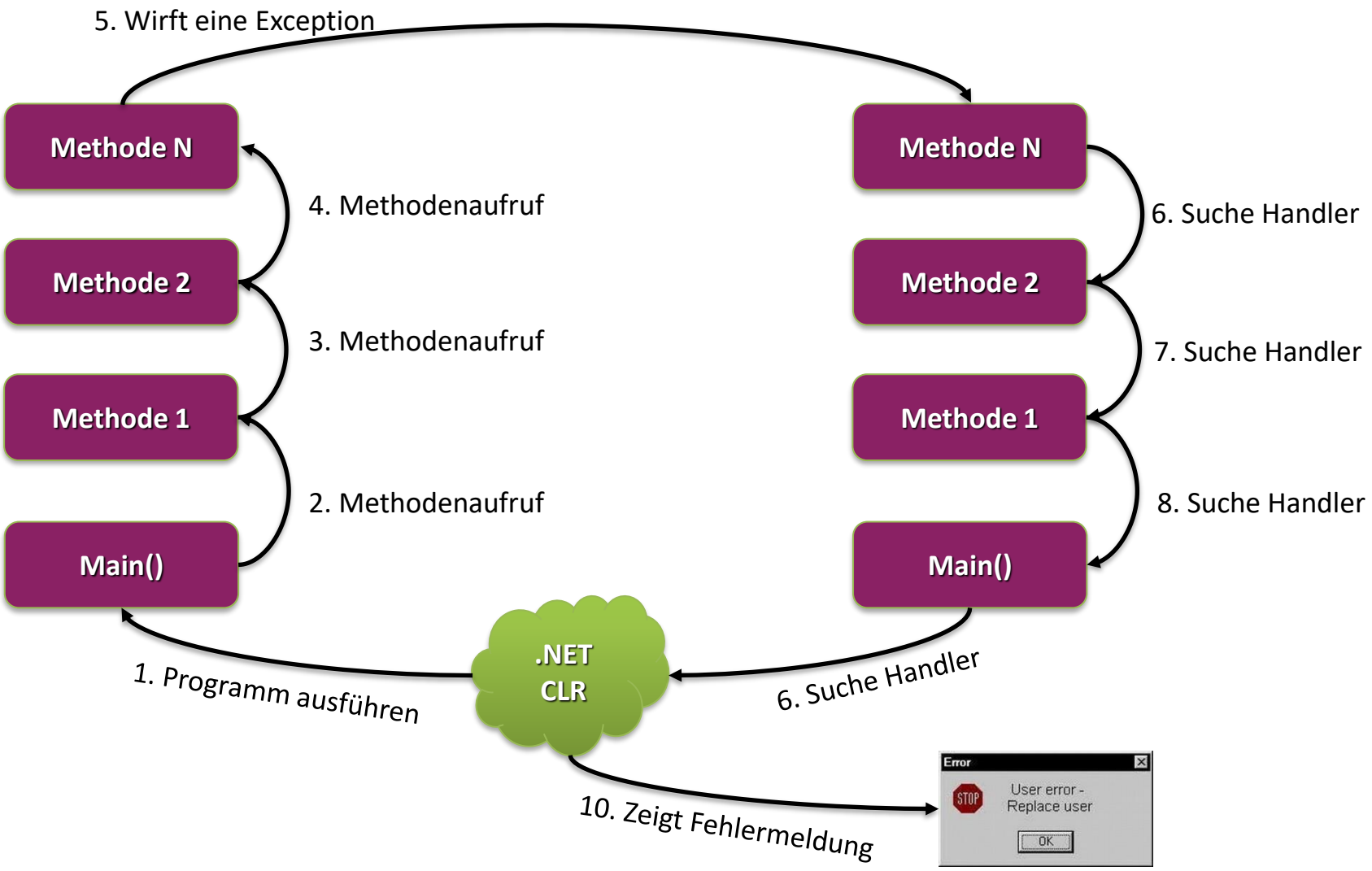
- Laufzeit- vs. fachliche Fehler

Hands On



Demo Ausnahme

Der Weg einer Exception



Try-catch-finally

- In C# können Ausnahmen mit dem **`try-catch-finally`**-Konstrukt behandelt werden
- Im **`try`**-Block ist der Code, der potenziell eine Ausnahme verursachen kann
- Der **`catch`**-Block fängt die Ausnahmen und kann mehrfach vorkommen für jeweils unterschiedliche Ausnahme-Typen
- Reihenfolge der catch-Blöcke ist entscheidend
  - Von spezieller Ausnahme zur Allgemeinen
- Falls ein Fehler nicht abgefangen wird, stürzt das Programm ab
- Der **finally**-Block wird definitiv immer ausgeführt unabhängig, ob eine Ausnahme geworfen wurde oder nicht

Try-catch-finally - Beispiel

```
static void Main(string[] args)
{
   string eingabe = Console.ReadLine();

   try
   {
     int zahl = int.Parse(eingabe);
     Console.WriteLine($"Sie haben eine gültige integer Zahl {zahl} eingeben");
   }
   catch (FormatException)
   {
     Console.WriteLine("Sie haben eine ungültige Eingabe eingeben");
   }
   catch (OverflowException)
   {
     Console.WriteLine("Sie haben eine ungültige integer Zahl eingeben. Die
                        Zahl ist zu groß für integer");
   }
   catch (Exception ex) when (ex.Message == "Bedingung") //C# 6 Exception Filter
   {
     Console.WriteLine($"Fehler {ex.Message}");
   }
   finally
   { Console.ReadLine(); }
}
```

Hands On

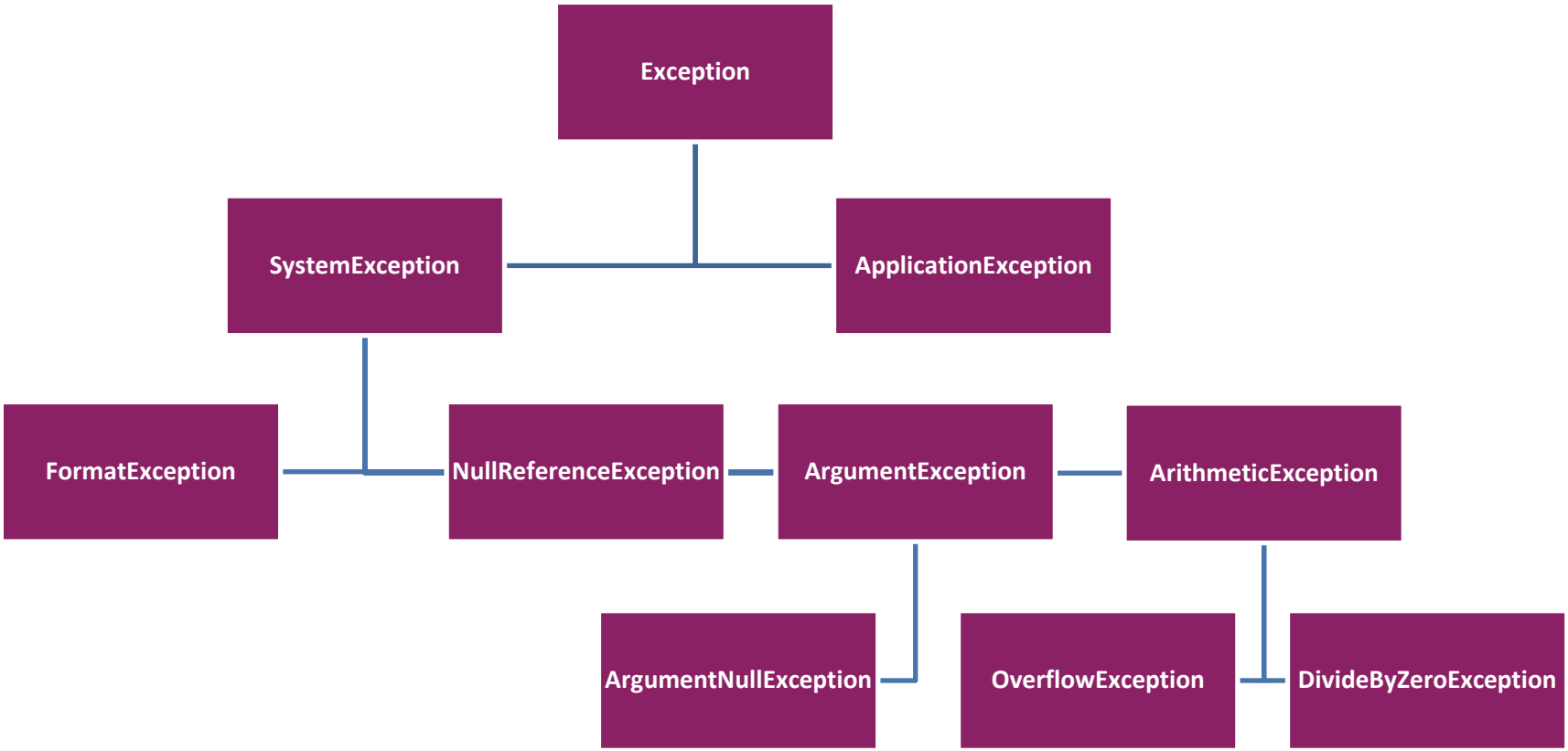


Demo Ausnahme mit try-catch-finally
& VS Debugging

System.Exception-Klasse

- Exceptions in .NET sind Objekte
- Die **`Exception`**-Klasse des .NET Frameworks ist die Basisklasse aller Fehlerklassen
- Sie beinhaltet Informationen über die Fehlerursache
  - **`Message`** – Textbeschreibung der Ausnahme
  - **`StackTrace`** – Snapshot vom Stack im Moment des Ausnahmewurfs
  - **`InnerException`**- Ausnahme welche aktuelle Ausnahme verursacht hat (falls vorhanden)
- Möglichkeit benutzerdefinierte Exceptions zu erstellen

Exceptions-Hierachie (Auszug)



Die 4 von der Exceptions-Tankstelle

NullReferenceException	• Zugriff auf ein Objekt, das **null** bzw. nicht initialisiert ist
FormatException	• Fehler bei Umwandelungen von Datentypen (z.B. string in int)
ArgumentException	• Problem mit übergebenen Parameter an Methode (z.B. Format, Gültigkeitsbereich)
ArgumentNullException	• Übergabe von **null** bzw. nicht initialisiertes Objekt an die Methode

Exception werfen

- Exception mit Fehlermessage

```
throw new ArgumentException("Invalide Summe");
```

- Exception mit Fehlermessage und InnerException

```
try
{
  int.Parse(eingabe);
}
catch(FormatException ex)
{
  throw new ArgumentException("Ungültige Nummer",ex);
  //throw ex; //Exception kann widerum geworfen werden
  //throw; //Abkürzung -> letzte Exception wird geworfen
}
```

Eigene Exception-Klasse

- Ableitung von Exception
  - Konvention Suffix „**`Exception`**“ für Klassenname
  - 4 Kontruktoren von Exception + evtl. eigene
  - Klasse als [**`Serializable`**] markieren

```
[Serializable]
public class MeineException : System.Exception
{
    public string FachlicheErrorMessage { get; set; }
    public int TechnischeErrorCode { get; set; }
    public MeineException(string fachlicherErrorMessage, int technischerErrorCode)
    {
       FachlicheErrorMessage = fachlicherErrorMessage;
       TechnischeErrorCode = technischerErrorCode;
    }

    public MeineException()  {}
    public MeineException(string message) : base(message) {}
    public MeineException(string message, System.Exception inner) : base(message, inner) {}
    protected MeineException(SerializationInfo info,StreamingContext context) : base(info, context)  {}
```

Hands On



Demo Ausnahme mit try-catch-finally Fortsetzung

Generics

Warum brauchen wir Generics?

„Generika führen das Konzept der Typparameter in .NET Framework ein. Dadurch wird des möglich, Klassen und Methoden zu entwerfen, in denen die Angabe eines oder mehrerer Typen verzögert wird, bis die Klasse bzw. Methode durch Clientcode deklariert und instanziiert wird.“

Quelle: MSDN

Hands On



Generics

Generische Klassen

- Übliche Klassendefinition

```
public class NormaleKlasse
```

- Definition für eine generische Klasse
  - **T** steht als Platzhalter, für den bei Verwendung angegeben wird, welcher Typ verwendet werden soll

```
public class GenerischeKlasse<T>
```

Generics Bedingungen/Einschränkungen

Einschränkung	Beschreibung
where T : struct	Der verwendete Parameter T muß ein beliebiger Werttyp sein. Nullfähige Werttypen dürfen nicht verwendet werden.
where T : class	Der verwendete Parameter T muss ein Referenztyp sein (Klassen, Schnittstellen, Delegaten und Arrays)
where T : new()	Der verwendete Parameter T muß über einen Standardkonstruktor verfügen (muß als letzte aufgeführt werden, wenn sie mit anderen Einschränkungen kombiniert wird)
where T : <Basisklasse>	Der Parameter T muß von der angegebenen Basisklasse abgeleitet worden sein
where T : <Schnittstelle>	Der Parameter T muss die angegebene Schnittstelle implementieren
where T : <U>	Der verwendete Parameter T muss selbst vom generischen Typ vererbt sein, der als Parameter U verwendet

Generics

- Constraints definieren

```
class GenKlasse<T>: where T : IComparable,IDisposable
```

- Standardwert – default
  - Wertetyp = 0, Referenztyp = null

```
T myT = default(T);
```

- Generische Methoden

```
public void DoStuff<T>(T obj);
```

Generics und Vererbung

- Vererbte Klasse kann generischen Typparameter der Basisklasse übernehmen oder konkreten Typ angeben

```
class BaseClass<T> where T : IDisposable
{}
```

```
class SubClass<T> : BaseClass<T>
where T : IComparable
{
  …
}
```

```
class SubClass : BaseClass<int>
{
  …
}
```

Ko-/Kontravarianz für generische Interfaces

- Kovarianz ermöglicht die Verwendung (Rückgabewert) eines stärker abgeleiteten Typs
  - Typ annotiert mit **out**-Schlüsselwort (z.B. IEnumerable)

```
IEnumerable<BasisKlasse> myObj = new List<AbgeleiteteKlasse>()
```

- Kontravarianz ermöglicht die Verwendung (Übergabeparameter) eines generischeren (weniger stark) abgeleiteten Typs
  - Typ annotiert mit **in** - Schlüsselwort

Regeln

- Generische Typen dürfen nur Methoden enthalten, die für alle Typen gelten
- für generische Typen können keine arithmetischen Operationen definiert werden
- in der Parameterdefinition dürfen keine nicht-generischen Parameter verwendet werden (z.B. <T, int> ist nicht erlaubt)
- Der Parametertyp einer generischen Klasse darf nicht gleichzeitig Basisklasse des generischen Typs sein
- Ein generischer Typ kann nicht selbst Parameter eines anderen generischen Typs sein
- Generische Typen können in die üblichen Vererbungsmechanismen eingebaut werden.

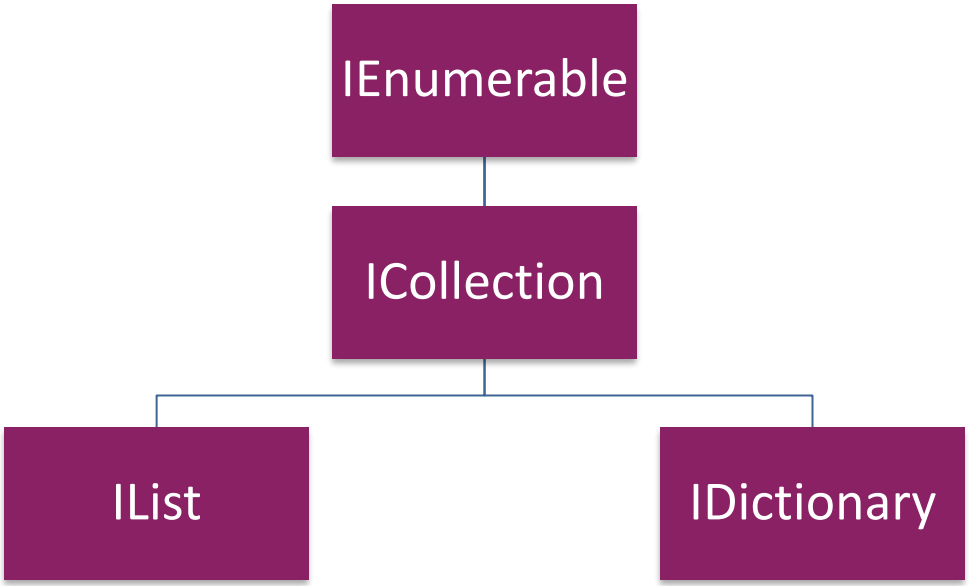
Hands On



Generics Demo +
„Müll“ Aufgabe

Collections

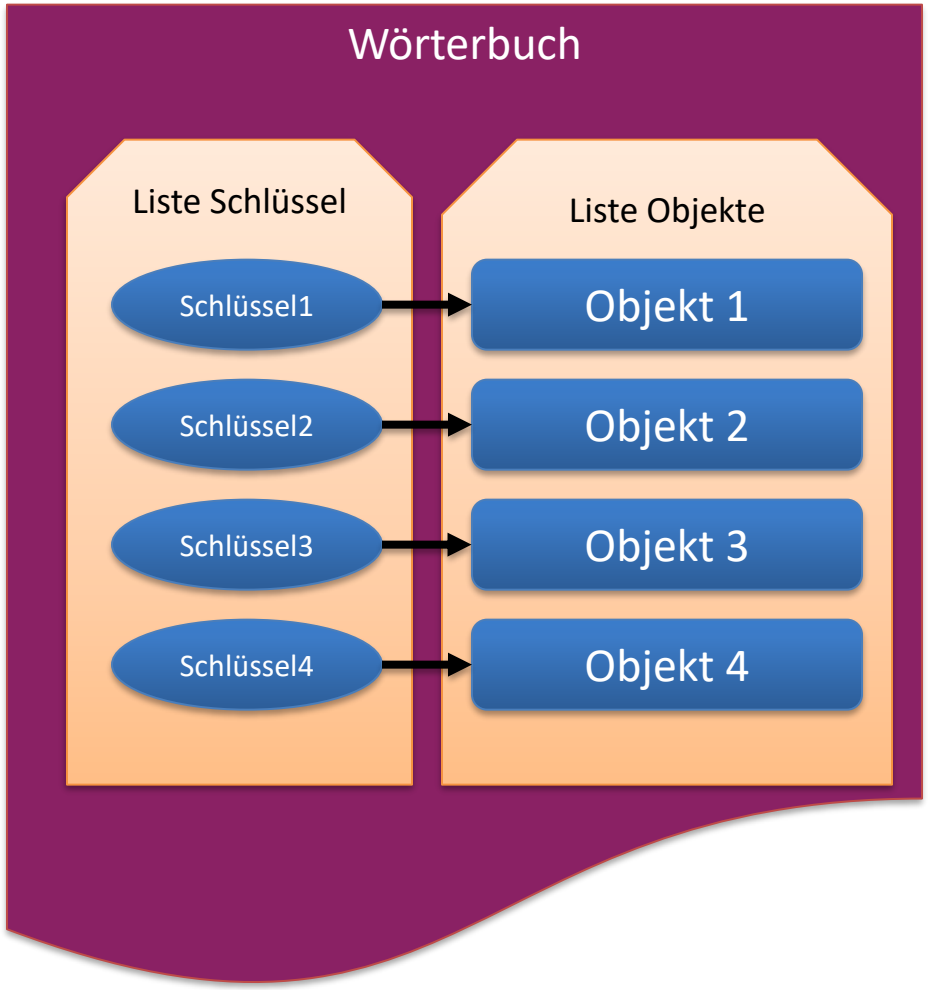
Vererbungszusammenhang



Bezeichnung	Beschreibung
IEnumerable	Basisklasse, die das Iterieren durch die Auflistung per foreach ermöglicht
ICollection	Basisklasse, die Grundeigenschaften und –methoden implementiert (z.B. Anzahl der Elemente), vor allem aber für Threadsicherheit sorgt
IDictionary	Basisklasse für alle Auflistungen, die die Objekte als Key-Value-Pair sammelt
IList	Basisklasse für alle arrayähnlichen (eindimensionale) Listen

Liste vs. Wörterbuch





Auflistungsarten

Bezeichnung	Beschreibung
ArrayList	einfache Auflistung von Objekten
SortedList	sortierte Liste von Key-Value-Pairs
Queue	Warteschlange, first-in, first-out
Stack	Stapel, last-in, first-out
Hashtable	Liste mit Key-Value-Pairs, bei der anhand von Schlüssel oder Index auf ein Element zugegriffen werden kann
BitArray	Auflistung von Boolean-Werten
StringCollection	Auflistung von Strings
StringDictionary	Key-Value-Pair Auflistung von Strings, Zugriff über Schlüssel oder Index
ListDictionary	für kleinere Listen mit Objekten
HybridDictionary	Auflistung, die je nach Größe die Objekte intern in einem ListDictionary oder in einer Hashtable speichert
NameValueCollection	Key-Value-Auflistung mit Strings, Zugriff über Schlüssel oder Index

Generische Auflistungsarten

Bezeichnung	Beschreibung
List<>	einfache Auflistung von Objekten
Queue<>	Warteschlange
Stack<>	Stapel
Dictionary<>	Auflistung mit key-value-Paaren
SortedList<>	sortierte Liste
SortedDictionary<>	sortiertes Wörterbuch
LinkedList<>	verkettete Liste

Eigene Auflistungen

- Man kann eine einfache eigene Auflistungsklasse erstellen, indem die eigentliche Auflistung, in der die Objekte gesammelt werden, in einer eigenen Klasse kapselt
- Man kann eigene Auflistungsklassen erstellen, indem man seine Klasse von einer der Basisklassen abgleitet und alle zugehörigen Eigenschaften und Methoden selbst implementiert



LinQ

Language **In**tegrated **Q**uery

Themen

- Die Idee hinter LinQ?
- LinQ Syntax
- Lambda-Expressions
- Erweiterungsmethoden
- Anonyme Datentypen

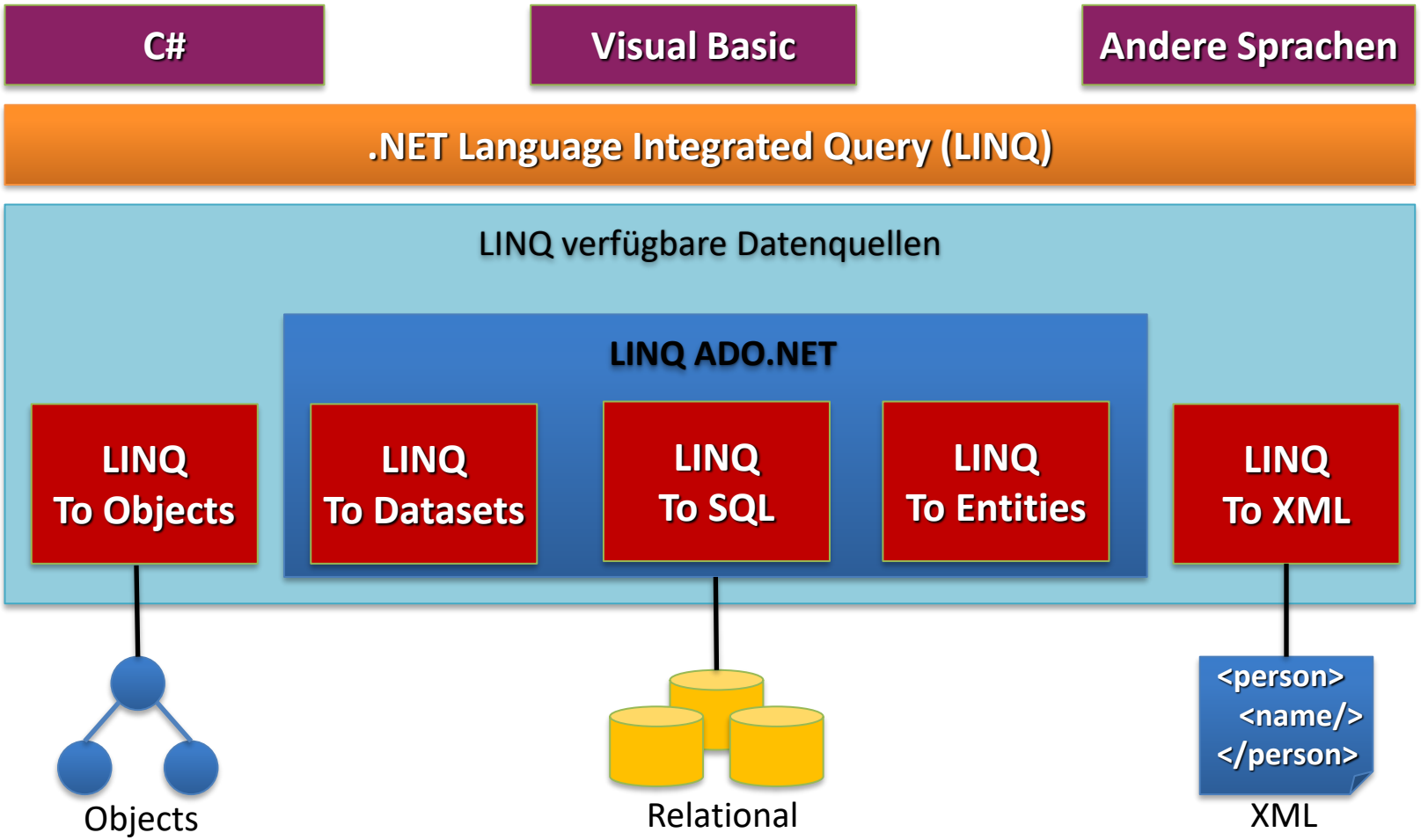
Die Idee hinter LinQ

- Viele verschiedene Datenquellen
  - DB, SAP, Exchange, Sharepoint, BizTalk, XML
- Viele Leistungsstarke Hochsprachen
  - C#, VB.NET, Java, Delphi
- Das Problem liegt in der Datenbeschaffung durch unterschiedliche Abfragesprachen / Syntax
  - Meist string basiert und untypisiert

LINQ – Die universal Abfragesprache

- Programmiermodell zur einheitlichen Abfrage von Datenquellen
- Imperativ => Deklarativ
  - Nicht ***wie*** man die Daten/Informationen beschafft, sondern ***was*** man haben möchte ist im Fokus
- Bietet eine „SQL“-ähnliche Syntax
- Streng typisiert
- Queries werden zur Laufzeit geprüft, nicht erst zur Ausführungszeit
- Lazy Loading
- Erweiterbares Provider-Konzept
  - Linq-to-Sharepoint, Linq-to-Amazon
- Ab .NET 3.5 verfügbar

LinQ Architektur



LinQ - Varianten

- LinQ to Objects
  - Arbeitet auf IEnumerable
- LinQ to XML
  - Arbeitet auf XML Strukturen
- LinQ to SQL
  - Arbeitet auf SQL-Datenbanken
- LinQ to Entities
  - ADO.NET Entity Framework

LinQ Syntax

```
static void Main(string[] args)
{
  int[] numbers = { 5,7,1,3,9,2,0,8,4,6};

  var lowNumbers = from n in numbers
                   where n < 5
                   orderby n descending
                   select n;

  foreach(var x in lowNumbers)
  {
    Console.WriteLine(x)
  }
}
```

„select“ wählt das Element oder Teilmenge fürs Ergebnis

n ist die „Laufvariable“ Steht für ein einzelnes Element in numbers (Collection)

„where“ ist die Filterung

„orderby“ ist die Sortierung

Anonyme Typen

- Dienen zum Gruppieren von Daten in ein Objekt, ohne das man explizit eine Klasse deklarieren muss

```
var obj = new { Name="Müller", Vorname="Max" };
```

- Intern entsteht ein anonymer Typ/Klasse
- Kommt häufig mit LinQ-Abfragen zum Einsatz

Hands On



LinQ

Lambda Expressions

- Mit Lambda-Ausdrücken wird das Einfügen von Code an stellen erlaubt, wo man eigentlich eine Methoden-Übergabe an ein Delegate erwartet hätte
- Lambda-Ausdrücke sind anonyme Methoden
- LinQ-Abfragesyntax wird beim Kompilieren in Lambda-Syntax übersetzt

LinQ Syntax II

Filtern von Daten

Where

Projektionsvorgänge

Select
SelectMany

Sortieren von Daten

OrderBy
OrderByDescending
ThenBy
ThenByDescending
Reverse

Partitionieren von Daten

Take
Skip
TakeWhile
SkipWhile

Aggregationsvorgänge

Count
Sum
Min
Max
Average
Aggregate

Elementvorgänge

First
FirstOrDefault
ElementAt
…

Quelle: MSDN + 101 LINQ Samples

Erweiterungsmethoden

- Mit Erweiterungsmethoden kann man bereits definierten Typen nachträglich Methoden hinzufügen
  - **`this`** kennzeichnet den zu erweiternder Typ
  - Klasse und Erweiterungsmethode müssen **`static`** sein

```
static class IrgendeineKlasse
{
  public static IEnumerable<TSource> Where<TSource>(
         this IEnumerable<TSource> source,
         Func<TSource, bool> predicate
)
```

Lambda-Ausdrücke

```
public static IEnumerable<TSource> Where<TSource>(
       this IEnumerable<TSource> source,
       Func<TSource, bool> predicate
)

static void Main(string[] args)
{
  Programmierer[] liste = GetListFromAnywhere();
  var csharpProgrammierer = liste.Where(
       programmierer => programmierer.Sprache == "C#")
  …
```

Eingabe-Parameter
Muss mit den Parametern des Delegates Func (TSource) entsprechen

Lambda-Operator

Ausdruck/Anweisungsblock
Rückgabetyp muss mit dem des Delegates (Boolean) übereinstimmen

Hands On



Lambda + Erweiterungsmethode

Delegates & Events

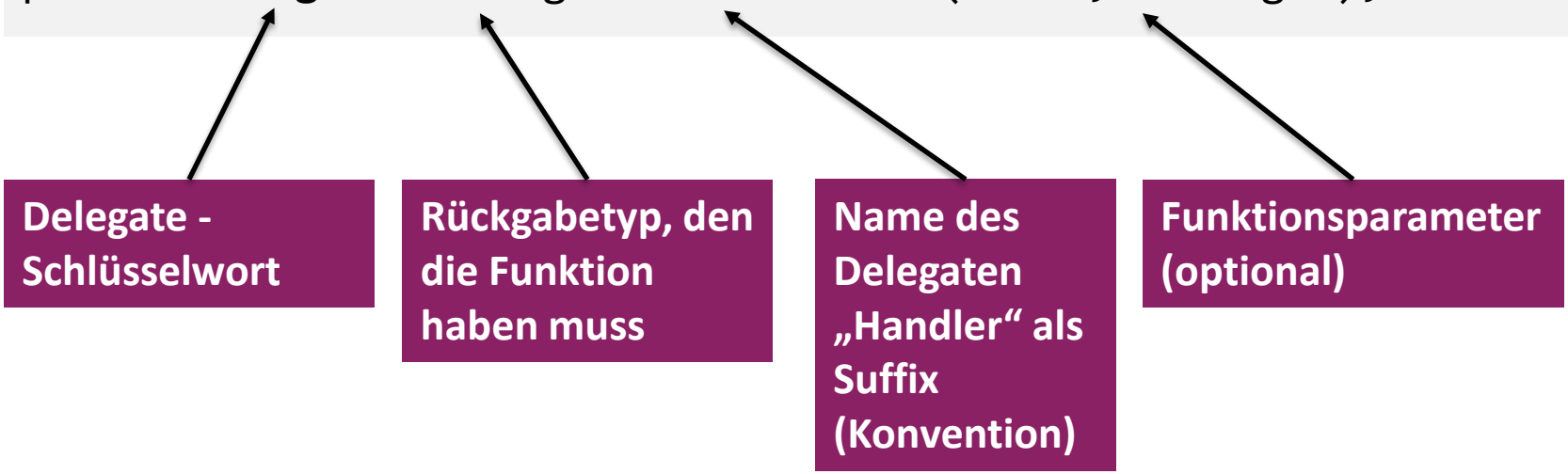
Themen

- Was ist ein Delegate?
- Multicast-Delegates
- Events & EventArgs

Was ist ein Delegate?

- Ein Delegate ist ein typisierter Funktionszeiger auf eine Methode
- Haupteinsatzgebiete sind LinQ & Events

```
public delegate string DoStuffHandler(int a, string b);
```



Hands On



Delegate – Einfacher Delegate + Demo Pumpe

Hinweise

- Einer Delegat-Variablen kann **`null`** zugewiesen werden
- Der Rückgabewert und die Methodenparameter werden durch den Delegaten bestimmt
- Bei Zuweisung des Delegaten darauf achten ***nur*** den Methodennamen zuzuweisen und ***nicht*** die Methode aufzurufen
- Möglichkeit auch anonyme Methoden statt klassischen Methodenbezeichner zu verwenden

Hands On



Delegate – Anonyme Methoden

Multicast Delegates

- .NET bietet die Möglichkeit mehrere Delegates zu einem einzigen zusammenzufassen
- += Operator fügt hinzu und -= entfernt aus der Aufrufliste des Delegaten

```
MyDelegate del = new MyDelegate(DoStuff);
del += new MyDelegate(DoStuffMore);

del += DoStuffIrgendEtwas //Kurzschreibweise

del();
```

Hands On



MultiCastDelegate – Demo Pumpen

Action<T> und Func<T> generische Delegates

- **Action<T>**
  - Der generische Action<T> - Delegate zeigt auf eine Methode **ohne** Rückgabewert (void)
    - **`Action<in T1, in T2 … in T16>`**

    (T1 .. T16 sind Eingabeparameter)

- **Func<T>**
  - Func<T> erlaubt Methodenaufrufe **mit** einem Rückgabewert
    - **`Func<in T1, in T2 … out TResult>`**

    (T1 … T16 sind Eingabeparameter und TResult ist Rückgabetyp)

Hands On

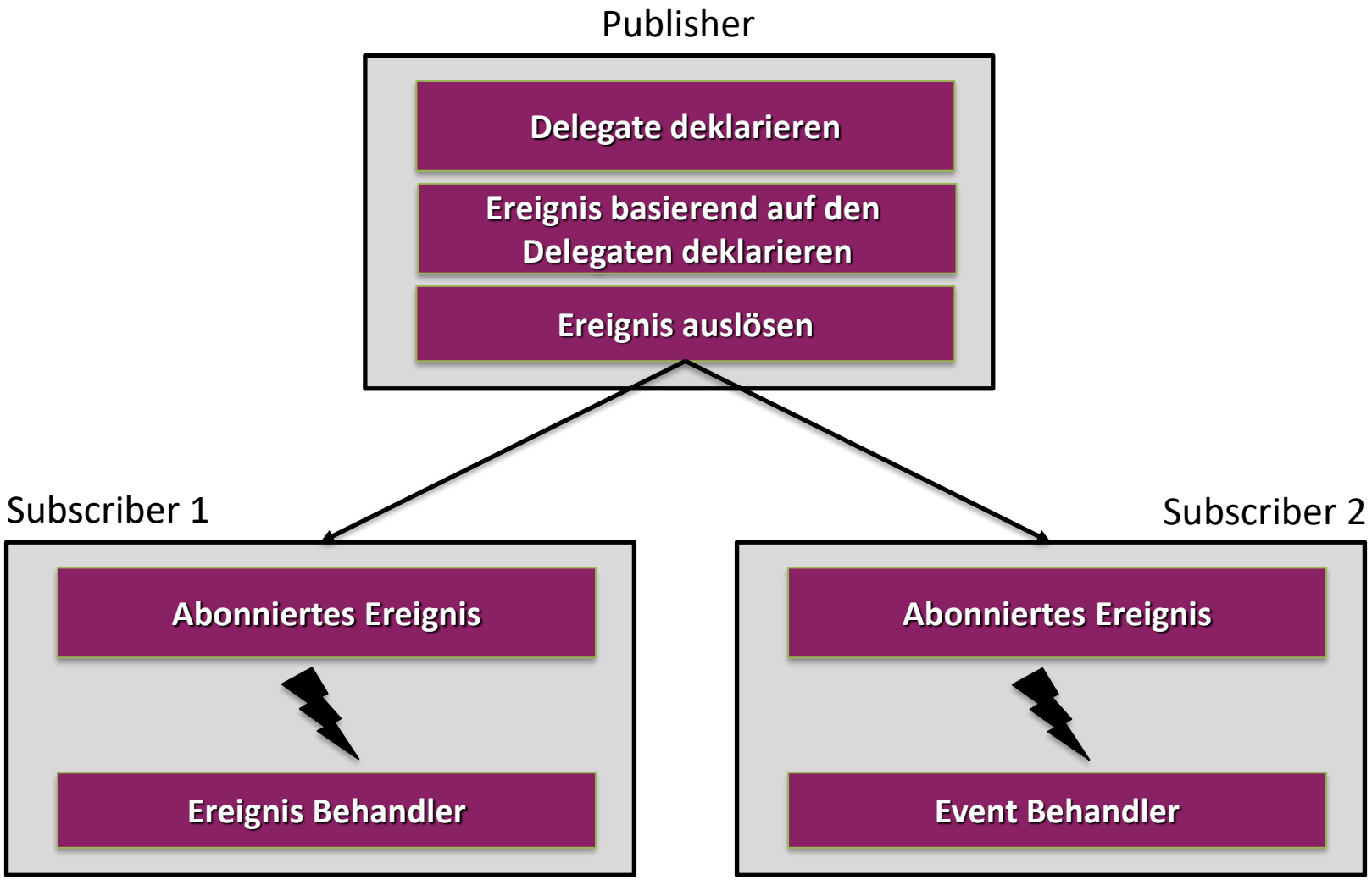


Generischer Delegate – Demo ActionUndFunc

Events

- Ereignisse (Events) befähigen Klassen bzw. Objekte andere Klassen/Objekte zu informieren wenn etwas geschieht
- Die Klasse, die das Ereignis auslöst ist der **publisher** und Klassen die das Ereignis empfangen/behandeln nennt man **subscriber** (Abonnent)
- Events basieren auf Delegates
- Events können von jedem Delegat-Typ angelegt werden, der **`void`** als Rückgabewert hat

Events


Publisher
Delegate deklarieren
Ereignis basierend auf den Delegaten deklarieren
Ereignis auslösen
Subscriber 1
Abonniertes Ereignis
Ereignis Behandler
Subscriber 2
Abonniertes Ereignis
Event Behandler

Events

- Definition eines Events

```
[<Zugriffsmodifizierer>] event <DelegateTyp> <EventBezeichner>

public delegate void PumpenHandler();
public event PumpenHandler OnKernUeberhitzung;
```

- Abonnieren eines Events

```
OnKernUeberhitzung += new PumpenHandler (ep.StarteElekPumpe);
```

- Benachrichtigung der Abonnenten

```
if (OnKernUeberhitzung != null)
    OnKernUeberhitzung();
```

Hands On



Events - Demo Pumpen 2

EventHandler & EventHandler<T>

- Vordefinierte Delegate

```
public delegate void EventHandler(object sender, EventArgs e);
public delegate void EventHandler<TEventArgs>(object sender,
TEventArgs e);
```

- Übergabeparameter werden mit abgeleiteter Klasse von EventArgs übergeben

```
public class MyEventArgs : EventArgs
{
   public string Message {get; set;}
}
```

Hands On



EventArgs - Demo Bar

C# 10 / 11 / 12 / 13 / 14

Exkurs

Implicit & Global Using Statements

- Vermeidet wiederholte Auflistung von Usings
- Aktivierbar in Projektdatei (Default-Einstellung)
  `<ImplicitUsings>enable</ImplicitUsings>`
- Globale Usings um Namespace projektweit bekannt zu machen

```
//Vorher
using System;
using System.Collecions.Generic;
using Microsoft.AspNetCore.Http

//Neu mit Implicit Using Statements
-

//Neu mit Global Using Statements in separater CS-Datei
global using Microsoft.AspNetCore.Http
```

File Scoped Namespaces

- Einrückung der Namespaces
- Konfigurierbar für gesamte Solution mit .editorconfig

```
//.editorconfig
csharp_style_namespace_declarations = file_scoped
dotnet_diagnostic.IDE0161.severity = suggestion
```

```
namespace MeinNamespace.Services
{
    public class MeineKlasse
    {}
}

//File Scoped Namespaces
namespace MeinNamespace.Services;

public class MeineKlasse
{}
```

MaxBy/MinBy & DateOnly and TimeOnly

- **DateOnly** bzw. **TimeOnly** repräsentieren entweder das Datum **oder** die Uhrzeit im Gegensatz zu DateTime
- **MinBy/MaxBy** geben ganzes Objekt zurück

```
//DateOnly & TimeOnly
DateOnly date = DateOnly.MinValue; //01.01.0001 ohne Zeit
TimeOnly time = TimeOnly.MinValue; // 12:00

//MaxBy/MinBy
List<Person> people = new List<Person>
{
  new Person { Name = "John Doe", Alter = 25},
  new Person { Name = "Jane Doe", Alter = 23}
}

var person = people.MaxBy(c => c.Alter); // John Doe
```

Raw String Literal / Required Member

- Unformatierte Zeichenfolgen-Literale können beliebigen Text enthalten, ohne dass Escapezeichen erforderlich sind
- Required – Modifizierer gibt an, dass das Feld / Eigenschaft von allen Konstruktoren oder via Objektinitializer initialisiert werden muss

```
//String Literal vor C#11
var message = "Der folgende Text ist sehr \"wichtig\" ".

//ab C#11
var message = """Der folgende Text ist sehr "wichtig" """.


//Required Member
public required string FirstName {get; init;}
```

Primärkonstruktoren C# 12

- Verkürzte Schreibweise von Konstruktoren. Achtung parameterloser Konstruktor ist nicht mehr vorhanden und muss separat deklariert werden. Dadurch Aufruf des Primärkonstruktors

```
public class Kunde(Guid kundeId, string name, float preis)
{
    public override string ToString()
    {
     return $": {kundeId} {name} {preis}";
    }
}
```

Vereinfachte Initialisierung von Mengen, Spread, Opt. Parameter in Lambdas C# 12

- Syntax mit eckigen Klammern wie in Javascript
- Spread-Operator möglich (Array aus anderen Arrays)

```
// Vorher
string[] fruechte = new string[] {"Banane", "Orange"};

//Nachher
string[] fruechte = ["Banane", "Orange"];

//Spread-Operator
string[] alleFruechte = [.. fruechte, "Apfel"];
```

Params Collection C# 13

- Der „params“-Parameter ist nicht mehr nur limitiert auf Array Typen sondern mit jedem Sammlungstyp nutzbar

```
// Vorher
public void MeineMethode(params string[] werte)


//Nachher
public void MeineMethode(params IEnumerable<string>
werte)
```

Extensions C# 14

- Erweiterungsmitglieder für Instanzmethoden, (statische) Instanzeigenschaften, statische Methoden

```
public static class MyExtensions
{
 //Vorher
 public static string Truncate(this string s) {…}

 //Nachher
 extension(string s)
 {
   public string Truncate() {…}
   public bool IsEmpty => string.IsNullOrEmpty(s);
   public static string Create(int count) {..}
 }
}
```

HttpClient

Http-Client

- Die Klasse `HttpClient` bietet die Möglichkeit der Konsumierung von Web-Api-Services für .NET-basierte Clients
- `Asynchrones Programmiermodell`

HttpFactory & TypedClient

- Möglichkeit den Http-Client durch IHttpClientFactory zu erstellen (Empfehlung)

```
builder.Services.AddHttpClient("MeineAPI", opt =>
{
    opt.BaseAddress = new Uri("https://localhost:5011/api/");
});

//---------- Komponente ----------------------------------------------------------
private IHttpClientFactory HttpClientFactory { get; set; }

protected override async Task OnInitializedAsync()
{
    var client = HttpClientFactory.CreateClient("MeineAPI");
}
```

Typed HttpClient

- Weitere Möglichkeit den Http-Client als TypedClient zu nutzen

```
builder.Services.AddHttpClient<IMeinService,MeinService>(opt =>
{
    opt.BaseAddress = new Uri("https://localhost:5011/api/");
});

//---------- MeinService.cs ------------------------------------------------------
public class MeinService
{
    private readonly HttpClient _client;

    MeinService(HttpClient client)
    {
        _client = client;
    }
}
```

Http-Client Methoden (Auszug)

Methode	Beschreibung
GetAsync(string requestUri)	Sendet einen GET-Request an die Uri
GetStringAsync(string requestUri)	Sendet eine GET-Request an den angegebenen Uri und gibt den Antworttext als Zeichenfolge zurück
PatchAsync(string requestUri, HttpContent content)	Sendet einen PATCH-Request an die Uri mit Content
PostAsync(string requestUri, HttpContent content)	Sendet einen POST-Request an die Uri mit Content
PutAsync(string requestUri, HttpContent content)	Sendet einen POST-Request an die Uri mit Content

Erweiterungsmethode	Beschreibung
GetFromJsonAsync<T>(string requestUri)	Sendet einen GET-Request an die Uri und deserialisiert die Antwort als JSON-Objekt
PostAsJsonAsync<T>(string requestUri, T value)	Sendet einen POST-Request an die Uri mit serialisiertem Content als JSON im Body
PutAsJsonAsync(string requestUri, T value)	Sendet einen PUT-Request an die Uri mit serialisiertem Content als JSON im Body

HttpClient - Beispiel

```
static void Main(string[] args)
{
 var url = "https://localhost/api/qotd";
 var client = new HttpClient();
 var response = await client.GetAsync(url);

 response.EnsureSuccessStatusCode();

 var content = await response.Content.ReadAsStringAsync();
 var quoteDto = JsonSerializer.Deserialize<QotdDto>(content);

 System.Console.WriteLine($"Spruch Rückgabe (Auszug) ->
 Name: {quoteDto.AuthorName} ## Beschreibung:
{quoteDto.AuthorBeschreibung} ##  Spruch:
quoteDto.QuoteText}");

  Console.ReadLine()
}
```

DEMO

„Http-Client“

Exkurs: Entity Framework Core

ADO.NET Entity Framework

- ORM-Framework
  - Bildet Objekte auf eine relationale DB ab
  - Blazor, Web Forms, MVC, WPF, WCF, Web API
  - Unabhängig von verwendeter Datenbank





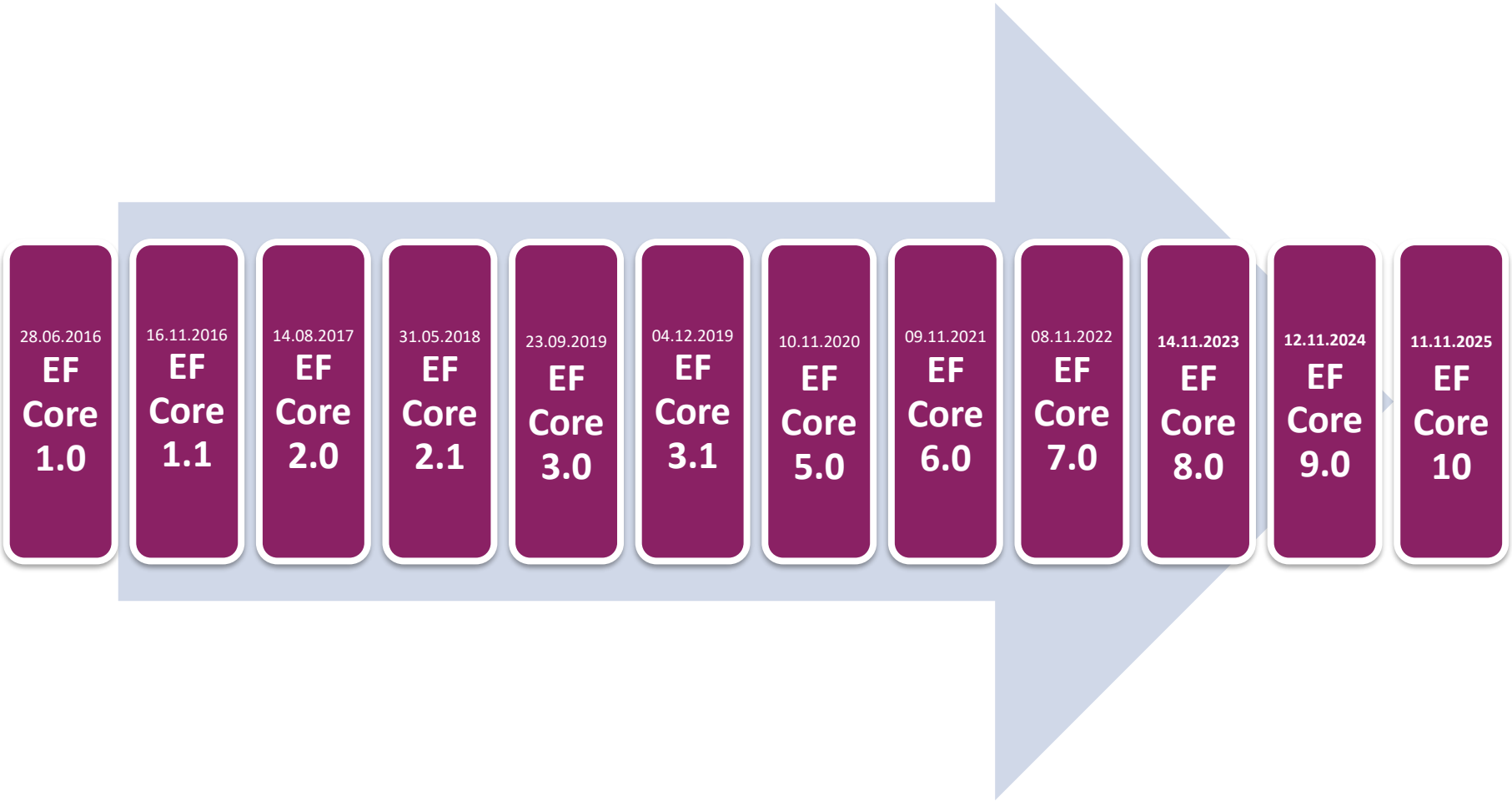








Versionshistorie Entity Framework



Ohne Entity Framework

- Fehleranfällig, Unsicher, Kompliziert

```
string con = "Data Source=.\\;Initial Catalog=AutorDB;IntegratedSecurity=True";
string cmd= "INSERT INTO Author (LastName,FirstName) VALUES (‘Twain', ‘Mark')";

using (SqlConnection connection = new SqlConnection(connString))
{
   using(SqlCommand com= new SqlCommand(cmdString))
   {
      com.Connection= con;
      connection.Open();
      com.ExecuteNonQuery();
   }
}
```

Mit Entity Framework

- Objekte <> Tabellen Abbildung
- Schutz vor SQL-Injections
- Vermeidung von Syntaxfehlern
- ConnectionString in Konfigurationsdatei

```
using (var db = new AuthorDBContext())
{
   db.Authors.Add(new Author { LastName = "Twain", FirstName = "Mark" });
   db.SaveChanges();
}
```

EF Begrifflichkeiten

- Domain-Model
  - Konzeptionelles Model, welches ein Geschäftsproblem darstellt, aber noch keine technische Lösung

- Objekt – Modell
  - Eine Klasse, die die Eigenschaften eines realen oder abstrakten Objektes beschreibt

Aufbau Entity Framework

- Entity
  - = Tabelle in Datenbank
  - Eigenschaften = Spalten der Tabellen
- Relationship
  - Fremdschlüsselbeziehungen in der Datenbank
- Context
  - Repräsentiert den Zugriff auf die Datenbank
  - CRUD

Model Workflows EF Core

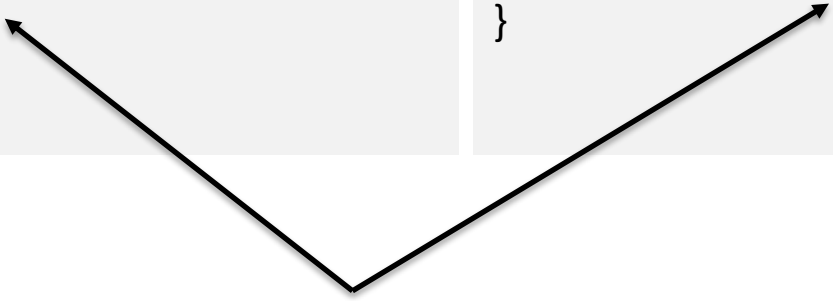
Vorgehensweise	Datenbank vorhanden	Beschreibung
Code First	**Nein**	Objekte und deren Beziehungen werden in Klassen beschrieben. Daraus wird dann mit dem EF die DB erstellt
Reverse Engineering Code First	**Ja**	Generierung von Entity Framework Klassen aus der DB

Code First Entitäten

- Für jede Entity eine Klasse erstellen

```
public class Author
{
   public Guid AuthorId { get; set; }
   public string Name { get; set; }
   public string Description { get; set; }
   public virtual List<Quote> Quotes { get; set; }
}
```

```
public class Quote
{
   public Guid QuoteId { get; set; }
   public string QuoteText { get; set; }
   public int AuthorId { get; set; }
   public virtual Author Author { get; set; }
}
```



Beziehung 1 : n

Code First Data Context

- Erstellen eines EF Data Context
- Data Context => DB
- Jede Entity **T** eine Eigenschaft **DbSet<T>** hinzufügen

```
public class QuoteContext : DbContext
{
    public QuoteContext(DbContextOptions<QuoteContext> options) :
                                base(options)
    {}

    public DbSet<Author> Authors => Set<Author>();

    public DbSet<Quote> Quotes => Set<Quote>();
}
```

Verwenden EF in Controller

- DataContext instanziieren
- Gewünschte Entitäten abfragen (Filtern möglich)
- Als Model an View übergeben

```
public partial class Index
{
    [Inject]
    private QuoteContext db {get; set;}

    public IEnumerable<Author> Authors { get; set; }

    protected override async Task OnInitializedAsync()
    {
        Authors = await db.Authors.ToListAsync();
    }
}
```

Database Migrations

- Löst das Problem Code und Datenbank synchron zu halten
- Möglichkeit von Snapshots
  - Generierung von Scripten um Code/Datenbank zu synchronisieren
  - Migrierung & Scripting SQL diverser Snapshots
- NuGet-Konsole
  - **`Add-Migration`** fügt neue Migration ein
  - **`Update-Database`** synchronisiert Modell mit Datenbank
  - **`Script-Migration`** erstellt SQL-Script

EF Core Commands

Installation des Tools
dotnet tool install --global dotnet-ef

https://docs.microsoft.com/de-de/ef/core/miscellaneous/cli/dotnet

Package Manager Console	dotnet CLI	Beschreibung
add-migration <migrationsname>	**dotnet ef migrations add <migrationsname>**	Erstellt eine Migration mit Migrations Snapshot
Remove-migration	**dotnet ef migrations remove**	Entfernt die letzte hinzugefügte migration
Update-database	**dotnet ef database update**	Aktualisiert die
Script-migration	**Dotnet ef migrations script**	Generiert ein SQL Script mit allen Migrationen

Hands On



Entity Framework

ADO.NET

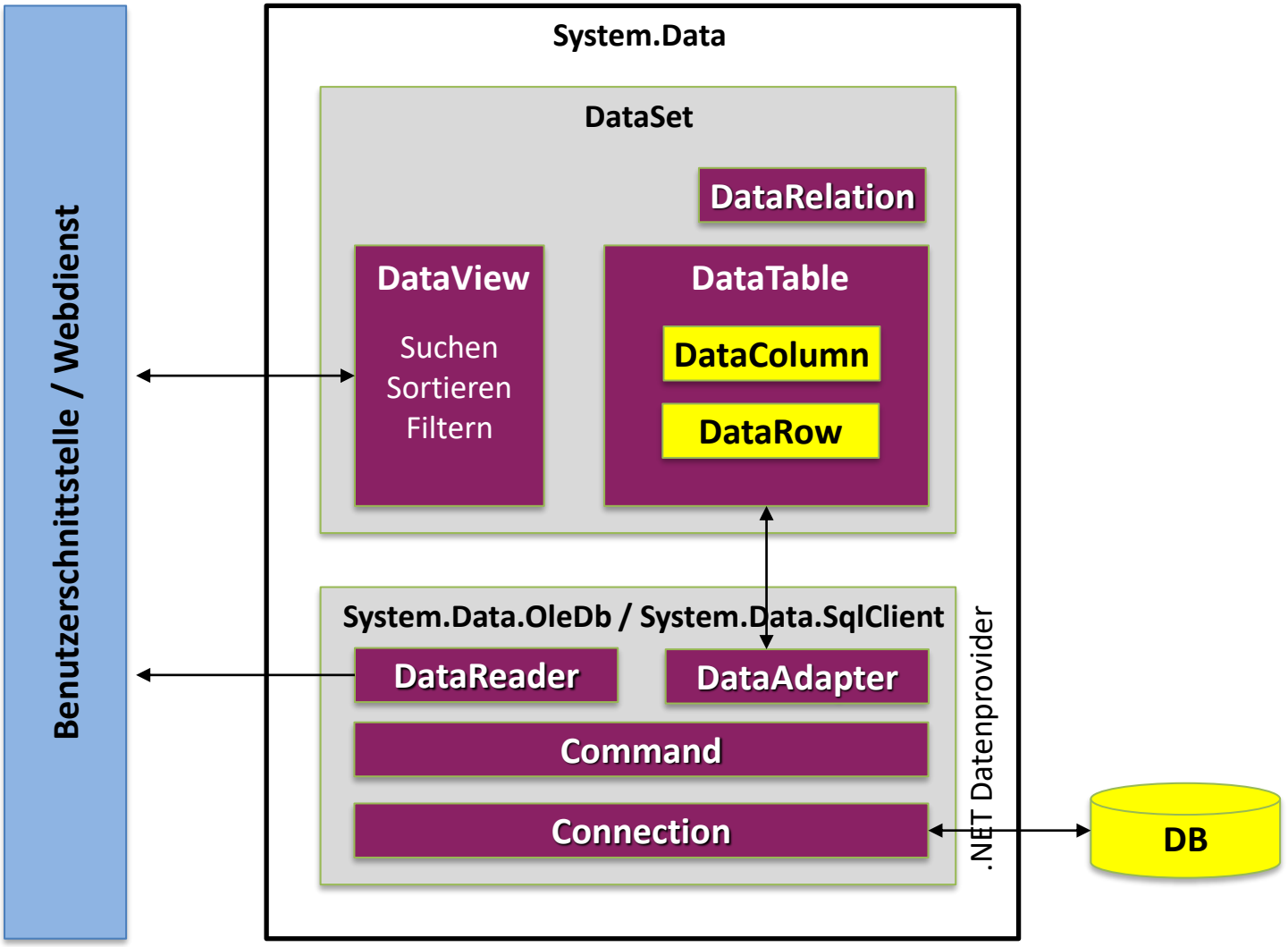
Themen

- Architektur ADO.NET
- Objektmodell
- (Typed) DataSets
- Entity Framework

Überblick ADO.NET

- Datenzugriffstechnologie für das .NET Framework
- Ziel der *unverbundenen* Datenbindung
  - ADO war für dauerhafte Verbindung konzipiert
- Kann mit relationalen und nicht relationalen Datenquellen interagieren

Architektur



Objektmodell

Klasse	Beschreibung	Enthalten in Auflistung
DataSet	Kern von ADO.NET und kann Container für alle anderen Objekte sein	
DataTable	Datentabelle, bestehend aus Zeilen (Row) und Spalten (Columns)	Tables
DataRow	Eine Zeile einer Datentabelle	Rows
DataColumn	Eine Spalte einer Datentabelle	Columns
Constraint	Definiert Einschränkungen innerhalb einer Datentabelle	Constraints
DataRelation	Beziehung zwischen Datentabellen	Relations
DataView	Sicht auf eine Datentabelle z.B. für Sortieren und Suchen	

Objektmodell

Klasse	Beschreibung
Connection	Verbindungsherstellung zur Datenquelle
Command	Ausführen von Befehlen
DataReader	Liefert die Ergebnisse einer Abfrage
DataAdapter	Verbindet das Dataset mit der Datenquelle zum Abrufen / Modifizieren der Daten

DataSet

- Lokaler Cache für Daten (disconnected)
- Universeller Datencontainer
  - Nicht nur zur Verwendung mit Datenbanken
  - XML lesen / schreiben
- Beinhaltet Tabellen und Relationen

Hands On



DataSet

Connection

- Das Connection-Objekt repräsentiert eine Verbindung zu einer Datenquelle

```
using System.Data;
using System.Data.SqlClient;

SqlConnection con = new SqlConnection(connectionString);
con.Open();
```

Command

- Repräsentiert eine Abfrage (CRUD) einer Datenquelle

Eigenschaft/Methode	Beschreibung
CommandText	Query bzgl. Datenbank
CommandType	Art des CommandTexts (Text, Stored Procedure)
ExecuteNonQuery()	Methode um Aktionsbefehle UPDATE, INSERT oder DELETE direkt gegen die DB auszuführen. Rückgabe ist die Anzahl der betroffenen Datensätze
ExecuteScalar()	Rückgabe der ersten Spalte der ersten Zeile aus der Menge der zurückgegebenen Datensätze (z.B. Select count(*) from..)
ExecuteReader()	Erzeugung eines DataReader-Objekts auf Basis eines SELECT-Befehls

DataReader

- Schneller serverseitiger Vorwärtscursor um Daten zu lesen
- Beinhaltet Methoden um unterschiedliche Spaltentypen einzulesen

Eigenschaft/Methode	Beschreibung
Read()	Automatisches Weiterbewegen zum nächsten Datensatz (meistens innerhalb einer while-Schleife)
GetValue()	Rückgabe des Werts einer Spalte basierend auf dem Spaltenindex
GetOrdinal()	Rückgabe des Spaltenindexes basierend auf Spaltenname

DataAdapter

- Hat eine „Brückenfunktion“, d.h. wird dazu genutzt Daten ins DataSets zu transportieren bzw. um Datenquellen mit den geänderten Inhalten von DataSets zu aktualisieren
- Enthält 2 Methoden
  - Fill
  - Update

Typisierte DataSets

- Eine von der Klasse DataSet abgeleitete Klasse um objektorientiert auf Inhalte zuzugreifen
- Vermeidung von Fehlern schon während der Kompilierzeit
- Besteht aus einer XML-Schema-Beschreibung (.xsd-Datei) sowie der Beschreibung der Anordnung von Daten in der grafischen Ansicht

Hands On



Command +
Datareader +
DataAdapter +
Typisierte Datasets